AIN SHAMS UNIVERSITY

Faculty of Engineering
Electronics and Communications Engineering Department

GRADUATION PROJECT THESIS — PART II

Machine Learning Techniques for PAPR Reduction in Multicarrier Communication Systems

---

Project Team

Nour-Eldin Mohamed Mostafa
Youssef Samir Sleem Ahmed
Muhammad Mahmoud Muhammad
Reem Nader Saeed
Basel Mohamed Ramadan
Ahmed Walid Hassan
Yosef Ahmed Mohamed

Supervisor

Dr. Michael Ibrahim

A thesis submitted in partial fulfillment of the requirements for the degree of Bachelor of Engineering

Academic Year: 2025/2026

Declaration of Authorship

We, Nour-Eldin Mohamed Mostafa, Youssef Samir Sleem Ahmed, Muhammad Mahmoud Muhammad, Reem Nader Saeed, Basel Mohamed Ramadan, Ahmed Walid Hassan, Yosef Ahmed Mohamed, declare that this thesis titled “Machine Learning Techniques for PAPR Reduction in Multicarrier Communication Systems” and the work presented in it are our own. We confirm that this material, submitted for assessment on the programme leading to the award of Bachelor of Engineering, is entirely our own work. We have exercised reasonable care to ensure that the work is original and does not, to the best of our knowledge, breach any law of copyright; it has not been taken from the work of others except to the extent that such work has been cited and acknowledged within the text.

Signed:

Signed:

Signed:

Signed:

Signed:

Signed:

Signed:

Date:

Abstract

Acknowledgements

This project represents not only two semesters of dedicated engineering work, but the culmination of our undergraduate journey at Ain Shams University. It would not have been possible without the guidance and support of many individuals.

First and foremost, we express our deepest gratitude to our supervisor, Dr. Michael Ibrahim, for their patience, technical insight, and for steering us in the right direction whenever we encountered a challenge — whether mathematical, computational, or conceptual.

We are grateful to the faculty and staff at the Electronics and Communications Engineering Department for providing the resources and foundational knowledge required to tackle this research. A special thanks to all the professors who challenged us to think as engineers, not merely as students.

To our families: thank you for being our anchor. Your unwavering support, encouragement, and understanding during long nights and stressful deadlines meant more than words can express. We hope we have made you proud.

Finally, to our friends and fellow students — thank you for the shared study sessions, the laughter, and the moral support that carried us through.

Contents

List of Figures

List of Tables

CHAPTER 1

OFDM System Model and Classical PAPR Reduction Techniques

To establish the requisite mathematical framework for this thesis, this chapter provides a preliminary review of the Orthogonal Frequency-Division Multiplexing (OFDM) signal model and the Peak-to-Average Power Ratio (PAPR) problem.It also examines classical PAPR reduction techniques and their inherent limitations, thereby motivating the shift toward the machine learning approaches explored in subsequent chapters.

1.1 OFDM Signal Model

Orthogonal Frequency-Division Multiplexing divides a wideband channel into N narrowband, orthogonal subcarriers. Each subcarrier carries one modulated symbol from a constellation such as QPSK or M-QAM. The modulation and demodulation operations are efficiently implemented via the Inverse Fast Fourier Transform (IFFT) and Fast Fourier Transform (FFT), respectively.

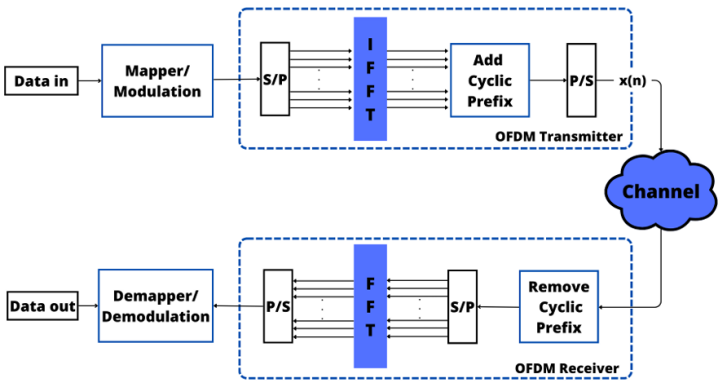


FIGURE 1.1: Block diagram of a baseband OFDM transmitter and receiver. The transmitter maps data onto N subcarriers via an N-point IFFT, adds a cyclic prefix (CP), and transmits the time-domain signal. The receiver removes the CP and applies an N-point FFT to recover the frequency-domain symbols [1].

1.1.1 Subcarrier Modulation via the IFFT

Let the frequency-domain data symbol vector after constellation mapping be

$$\boldsymbol{X} = \big[X_0,\ X_1,\ \ldots,\ X_{N-1}\big]^T,$$

where X_k is the complex data symbol mapped to the k-th subcarrier and N is the total number of subcarriers.

The time-domain OFDM signal is obtained by applying the N-point IFFT:

$$x[n] = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} X_k\, e^{\,j2\pi kn/N}, \qquad n = 0,\, 1,\, \ldots,\, N-1.$$

At the receiver, the original frequency-domain symbols are recovered by the complementary FFT operation:

$$X_k = \frac{1}{\sqrt{N}} \sum_{n=0}^{N-1} x[n]\, e^{-\,j2\pi kn/N}, \qquad k = 0,\, 1,\, \ldots,\, N-1.$$

The symmetric $1/\sqrt{N}$ normalization used in both of these transforms preserves signal energy between the frequency and time domains (Parseval's theorem).

1.1.2 Cyclic Prefix

Before transmission over the wireless channel, a **Cyclic Prefix (CP)** of length N_{cp} samples is prepended to each time-domain OFDM symbol by simply copying the final N_{cp} samples of the IFFT output and attaching them to the front. The CP serves two critical purposes:

- **ISI elimination:** provided N_{cp} exceeds the maximum channel delay spread, the CP completely absorbs all multipath echoes originating from the previously transmitted symbol.
- **Circular convolution:** the CP mathematically converts the linear convolution with the channel into a circular convolution, enabling simple single-tap equalization in the frequency domain via the FFT operation.

1.2 Peak-to-Average Power Ratio

1.2.1 Definition

The time-domain OFDM signal is the superposition of N independently modulated complex exponentials. When these subcarrier contributions align constructively at

certain time instants, large amplitude peaks occur. The severity of this phenomenon is quantified by the **Peak-to-Average Power Ratio (PAPR)**:

$$\mathrm{PAPR}(\boldsymbol{x}) = \frac{\max\limits_{0 \le n \le N-1} \left|x[n]\right|^2}{E\left[\left|x[n]\right|^2\right]}.$$

In this formulation, the numerator represents the peak instantaneous power and the denominator represents the average power of the OFDM symbol. The theoretical worst-case PAPR for N subcarriers is $10\log_{10}(N)$ dB, which occurs when all subcarriers align perfectly in phase.

1.2.2 CCDF Performance Metric

Since PAPR varies from one OFDM symbol to the next, it is characterised statistically using the **Complementary Cumulative Distribution Function (CCDF)**:

$$\mathrm{CCDF}_{\mathrm{PAPR}} = \Pr\big(\mathrm{PAPR} > \mathrm{PAPR}_0\big),$$

where PAPR_0 is a given threshold. For N subcarriers with independent, identically distributed symbols and Nyquist-rate sampling, the CCDF can be approximated as:

$$\Pr\big(\mathrm{PAPR} \ge \mathrm{PAPR}_0\big) \approx 1 - \left(1 - e^{-\mathrm{PAPR}_0}\right)^N.$$

A PAPR reduction technique shifts the CCDF curve to the left, indicating that high-PAPR events become statistically less likely.

1.2.3 Power Amplifier Nonlinearity

High PAPR has direct consequences for the **High-Power Amplifier (HPA)** at the transmitter. Every power amplifier has a saturation region: when the input signal exceeds a certain amplitude, the output clips or compresses, introducing nonlinear distortion. To avoid entering this nonlinear zone, the HPA must operate with sufficient **Input Back-Off (IBO)**:

$$\mathrm{IBO}\ (\mathrm{dB}) = 10\log_{10}\left(\frac{P_{\mathrm{sat}}}{P_{\mathrm{avg}}}\right),$$

where P_{sat} is the amplifier's saturation power and P_{avg} is the average input signal power. To prevent severe nonlinear distortion, the required IBO must be tightly bounded by the signal's maximum PAPR.

1.3 Classical PAPR Reduction Techniques

Classical PAPR reduction techniques can be broadly classified into **signal distortion** methods, which introduce controlled modifications to the transmitted signal, and **signal scrambling** methods, which generate alternative signal representations without distortion.

1.3.1 Signal Distortion Techniques

Clipping and Filtering

In this straightforward approach, any time-domain signal sample that exceeds a predefined amplitude threshold is deliberately clipped. Because this nonlinear operation inevitably introduces out-of-band spectral regrowth, the clipped signal must subsequently be filtered to meet spectral mask requirements, though this introduces some in-band distortion.

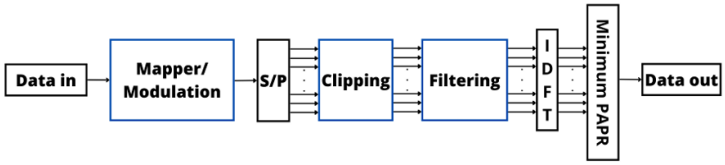


FIGURE 1.2: Block diagram of an OFDM transmitter with clipping and filtering. The signal is clipped in the time domain, then filtered in the frequency domain to remove out-of-band distortion [1].

While clipping and filtering is simple to implement and requires no side information, it introduces in-band distortion (increasing BER) and out-of-band radiation.

Tone Reservation (TR)

Tone Reservation sets aside a specific subset of subcarriers exclusively for generating peak-cancelling signals. This conceptually straightforward approach reduces PAPR without introducing any distortion to the actual data symbols, though it inevitably results in a slight reduction in overall spectral efficiency.

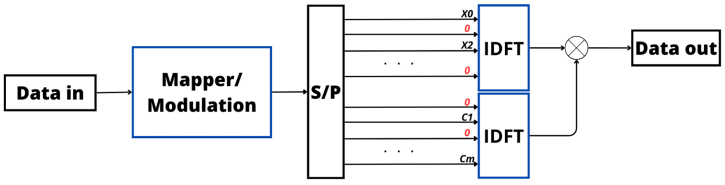


FIGURE 1.3: Block diagram of an OFDM transmitter with Tone Reservation (TR). A designated subset of subcarriers is reserved exclusively for generating peak-cancelling signals [1].

1.3.2 Signal Scrambling Methods

Signal scrambling methods alter the signal representation *without distortion* to find a low-PAPR version.

Partial Transmit Sequence (PTS)

The input data block is partitioned into disjoint sub-blocks, each transformed by an independent IFFT into the time domain. These sub-blocks are then individually weighted by different phase factors and summed together. The transmitter evaluates multiple phase combinations and transmits the one that yields the lowest overall PAPR.

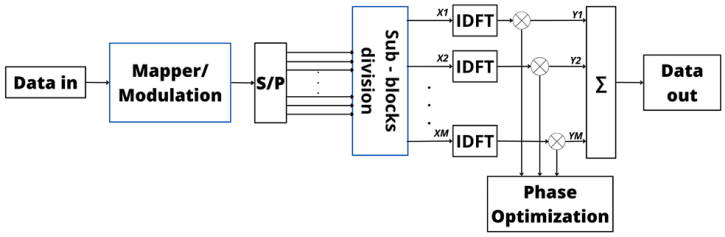


FIGURE 1.4: Block diagram of an OFDM transmitter with Partial Transmit Sequence (PTS). Data is split into M sub-blocks, each weighted by a phase factor after the IDFT. The combination with the lowest PAPR is transmitted [1].

Selected Mapping (SLM)

Selected Mapping generates multiple alternative representations of the same data block by multiplying the original frequency-domain symbols with several distinct, pseudo-random phase sequences. Each candidate sequence is transformed into the time domain via an independent IFFT, and the candidate exhibiting the absolute lowest PAPR is selected for transmission. To allow the receiver to correctly undo the phase rotation and recover the original data, the index of the chosen phase sequence must be transmitted alongside the signal as **side information (SI)**.

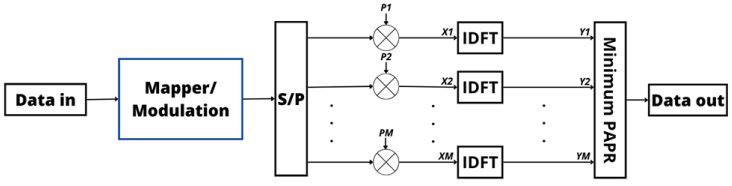


FIGURE 1.5: Block diagram of an OFDM transmitter with Selected Mapping (SLM). M different phase sequences are applied to the frequency-domain data before the IDFT. The candidate signal with the lowest PAPR is selected for transmission [1].

1.4 Limitations of Classical Approaches

Despite their ability to bound the peak power of OFDM signals, classical PAPR reduction techniques suffer from five fundamental limitations:

- **Distortion and spectral leakage:** Distortion-based methods, such as clipping and filtering, physically alter the time-domain waveform. This inevitably introduces in-band distortion that degrades the Bit Error Rate (BER) and causes out-of-band radiation that interferes with adjacent frequency channels.
- **Spectral and power inefficiency:** Methods like Tone Reservation (TR) avoid distortion by sacrificing a dedicated subset of subcarriers exclusively for peak-cancelling signals. This permanently reduces the system's spectral efficiency (data rate) and wastes transmission power on non-information-bearing signals.
- **Computational complexity:** Scrambling methods require multiple IFFT operations per OFDM symbol. For example, generating M candidate signals requires a per-symbol complexity of $\mathcal{O}(M \cdot N \log_2 N)$, which becomes prohibitive for large systems.
- **Signaling overhead:** Methods requiring side information (SI) must transmit $\lceil \log_2 M \rceil$ SI bits per symbol. These bits must be received error-free, necessitating additional channel coding that further reduces spectral efficiency.
- **Blind detection difficulty:** Eliminating SI overhead through blind detection at the receiver requires M full FFT operations per symbol, and classical decision metrics become highly unreliable at low Signal-to-Noise Ratios (SNR).

These limitations define the core design objectives for the machine learning-based approaches explored in subsequent chapters. By shifting from algorithmic search to data-driven optimization, deep learning models can achieve effective PAPR reduction while fundamentally overcoming the computational and signaling constraints of classical methods.

CHAPTER 2

PRNet: PAPR Reduction via Deep Autoencoder

The fundamental limitations of the classical PAPR reduction approaches identified in Chapter 1 motivate a paradigm shift in transceiver design. Rather than relying on rigid algorithmic searches or intentional signal distortion, Kim et al. [2] proposed the **PAPR Reduction Network (PRNet)**, the first deep learning-based system for joint PAPR and BER optimisation in OFDM. PRNet treats the transmitter–channel–receiver chain as a single deep autoencoder: the encoder learns a data-dependent constellation mapping that jointly minimises reconstruction error and PAPR, while the decoder learns the corresponding inverse mapping, fundamentally bypassing the computational and signaling bottlenecks of classical methods.

2.1 Autoencoder Framework

In a traditional communication system, the transmitter transforms raw data into a physical signal through a rigid sequence of fixed mathematical operations, which the receiver attempts to run in reverse after the signal is distorted by the wireless channel. This physical reality maps perfectly onto the architecture of a deep learning **autoencoder** [2]. In this framework, the neural network's *encoder* acts as the transmitter, the *decoder* acts as the receiver, and the mathematical layer connecting them acts as the noisy wireless channel. By placing a mathematical simulation of the channel directly between the encoder and decoder, the entire communication chain becomes a single, unified neural network.

The profound advantage of this unified architecture is that it enables **end-to-end joint training**. Rather than optimising the transmitter and receiver as isolated digital signal processing blocks, PRNet optimises them simultaneously. This allows the network to dynamically learn a custom, data-dependent constellation mapping at the encoder that directly minimises both the reconstruction error (BER) and the peak power (PAPR), while the decoder simultaneously learns the exact inverse mapping required to recover the data despite physical channel impairments [2].

To realise this, PRNet integrates two trainable Deep Neural Networks (DNNs) into the communication chain. At the transmitter, the encoder DNN is placed before the IFFT, replacing the standard constellation mapper. Instead of mapping each symbol to a fixed constellation point independently, the encoder employs a fully connected structure where every output subcarrier value is a function of the *entire* input block. By processing all subcarriers jointly, the encoder detects input patterns that produce high PAPR and compensates by adjusting all subcarrier amplitudes and phases to generate low-peak waveforms [2].

At the receiver, the decoder DNN is placed after the FFT to replace the standard constellation de-mapper. Mirroring the transmitter, it employs a fully connected structure to process the received OFDM block jointly. Due to joint training, the decoder's weights implicitly encode the encoder's mapping. Consequently, it reconstructs the original data from the equalised frequency-domain subcarriers without explicit side information, eliminating both the bandwidth overhead and severe failure modes associated with SI errors in classical methods [2].

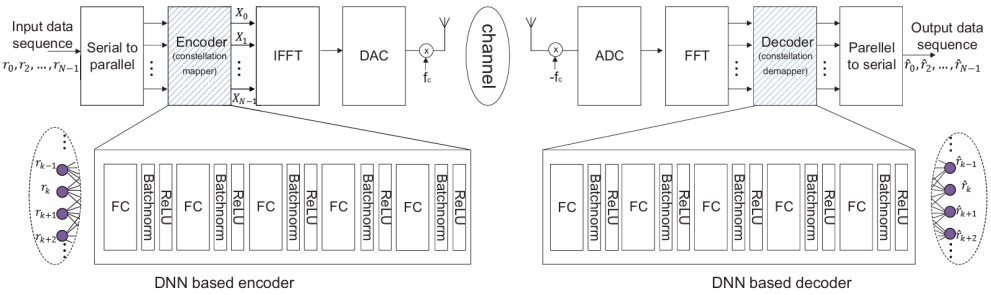


FIGURE 2.1: Structure of the PRNet system with DNN encoder and DNN decoder. Both consist of fully connected (FC) layers with Batch Normalization and ReLU activation. The encoder shapes the constellation before the IFFT to reduce PAPR; the decoder reconstructs the transmitted symbols after the channel and FFT [2].

2.1.1 Core Neural Network Components

Internally, both the encoder and decoder are constructed from identical repeated sub-blocks [2]. Each sub-block consists of three fundamental deep learning operations applied in sequence:

- **Fully Connected (FC) Layer:** Unlike standard OFDM where each subcarrier is mapped independently, the FC layers apply a linear transformation that combines the entire input vector. This dense connectivity enables the network to process all subcarriers jointly, recognising block-wide patterns that cause high PAPR and adjusting all subcarrier phases and amplitudes simultaneously.

- **Batch Normalization (BN):** Inserted after the linear transformations, BN standardises the intermediate constellation representations across each training batch. This stabilisation is critical for PRNet, as it prevents vanishing gradients and allows the network to converge on a highly complex, multidimensional constellation mapping without training collapse.
- **ReLU Activation:** The Rectified Linear Unit provides the essential nonlinearity. Without it, the entire multi-layer network would collapse into a single linear matrix projection, which is mathematically insufficient to discover the highly non-convex mappings required to jointly minimise PAPR and BER.

2.2 System Model

With the core neural network components established, we now formulate the PRNet transceiver as an end-to-end OFDM communication system [2]. The system is mathematically defined for N orthogonal subcarriers and an input sequence of M-bit symbols. For training and evaluation, we adopt the baseline configuration from Kim et al. [2], specifically $N = 64$ subcarriers and QPSK modulation ($M = 2$).

2.2.1 Input Representation

The original PRNet formulation [2] maps input sequences using 256-dimensional one-hot encoded vectors. This work replaces this categorical approach with a continuous real-valued in-phase and quadrature (I/Q) representation. Because standard deep neural networks operate exclusively in the real domain, they cannot process complex-valued OFDM symbols natively. To construct the necessary real-valued input, the 64 complex data symbols (each representing 2 bits) are explicitly decomposed into their in-phase (I) and quadrature (Q) components. The input sequence is thus defined as the 128-dimensional real-valued vector:

$$\boldsymbol{r} = [I_0, Q_0, I_1, Q_1, \ldots, I_{N-1}, Q_{N-1}]^T \in \mathbb{R}^{2N}.$$

While mathematically equivalent to the one-hot formulation in achieving optimal BER and PAPR, the continuous I/Q parameterization is adopted to provide three strict architectural advantages. First, it reduces the input dimension from 256 to 128, strictly halving the parameter complexity of the initial projection matrix. Second, bypassing categorical encoding eliminates discrete mapping layers, allowing the network to optimize continuous physical coordinate geometry directly. Finally, it enables explicit per-symbol power normalization during the forward pass. This guarantees the strict physical constraint $\mathbb{E}[|X|^2] = 1$ for every individual transmitted waveform, overcoming the reliance on global batch-wide statistical approximations.

2.2.2 DNN Encoder

The transmitter is parameterized by the deep neural network $f(\cdot;\theta_f)$, where θ_f denotes the complete set of trainable weights and biases. The encoder follows a topology of $L_f = 5$ consecutive hidden layers, each with a width of 2048 nodes [2]. Crucially, its final output layer is a pure linear projection with no non-linear activation or batch normalisation [2]. This is physically necessary because the network must map symbols to unconstrained coordinates on the complex I/Q plane, which inherently requires the ability to output unbounded negative and positive spatial values.

The encoder maps the real-valued input vector $\boldsymbol{r}$ directly to a frequency-domain vector representing the generated complex constellation points:

$$\boldsymbol{X} = f(\boldsymbol{r};\, \theta_f) = [X_0,\, X_1,\, \ldots,\, X_{N-1}]^T \in \mathbb{C}^N.$$

2.2.3 OFDM Modulation

The generated frequency-domain symbols X_k are modulated onto the $N = 64$ orthogonal subcarriers via an Inverse Fast Fourier Transform (IFFT) to produce the discrete time-domain sequence $x[n]$.

The Peak-to-Average Power Ratio (PAPR) of this time-domain sequence is mathematically defined as the ratio of its maximum instantaneous power to its average power:

$$\text{PAPR}\{x[n]\} = \frac{\max\limits_{0\le n\le N-1} |x[n]|^2}{\mathbb{E}[|x[n]|^2]}.$$

Since standard Nyquist-rate sampling ($L = 1$) often fails to capture the true continuous-time peaks, accurate evaluation requires an oversampling factor of $L \geq 4$. In practice, this is executed by zero-padding the frequency-domain vector $\boldsymbol{X}$ to length NL before computing the IFFT. This step ensures the discrete digital PAPR reliably bounds the maximum power of the physical analog waveform.

2.2.4 Channel Model

Following the IFFT operation, the time-domain signal is transmitted through a wireless channel before arriving at the receiver. As in the original PRNet formulation [2], the physical transmission is abstracted using a holistic operator $\mathbb{H}$ that models the combined effects of multipath fading and thermal noise.

2.2.5 OFDM Demodulation and Equalization

At the receiver, OFDM demodulation is executed by applying an $N = 64$-point Fast Fourier Transform (FFT) to convert the received time-domain sequence $y[n]$ back into the frequency-domain symbols Y_k.

Following the FFT, the transmission is modeled as a Rayleigh flat-fading environment. Under this model, the received signal on the k-th subcarrier is mathematically defined as $Y_k = H_k X_k + E_k$, where H_k represents the complex channel fading coefficient and E_k represents the additive white Gaussian noise (AWGN).

To compensate for the fading effects prior to neural network decoding, Zero-Forcing (ZF) equalization is applied. Assuming perfect Channel State Information (CSI)—meaning the receiver perfectly knows H_k—the equalized symbols are obtained by dividing the received signal by the channel coefficient:

$$\hat{Y}_k = \frac{Y_k}{H_k} = X_k + \frac{E_k}{H_k}.$$

2.2.6 DNN Decoder

The receiver is parameterized by the deep neural network $g(\cdot;\theta_g)$, where θ_g denotes the complete set of trainable weights and biases. The decoder shares a symmetric topology with the encoder, comprising $L_g = 5$ consecutive hidden layers, each with a width of 2048 nodes [2]. Similarly, its final output layer is a pure linear projection with no non-linear activation. This is required to allow the network to reconstruct unconstrained I/Q coordinate values from the noisy equalized signal prior to hard-decision demodulation.

The decoder maps the equalized frequency-domain vector $\hat{\boldsymbol{Y}}$ back to the 128-dimensional real-valued reconstructed sequence $\hat{\boldsymbol{r}}$. A hard-decision demodulator is subsequently applied to $\hat{\boldsymbol{r}}$ to recover the original transmitted bits.

Consequently, the complete end-to-end reconstructed symbol vector at the receiver can be written as a composite function of the entire transceiver pipeline:

$$\hat{\boldsymbol{r}} = g \circ \text{FFT} \circ \mathbb{H} \circ \text{IFFT} \circ f(\boldsymbol{r}),$$

where $\mathbb{H}$ abstractly denotes the physical effects of the wireless channel. Since every block in this composite chain is mathematically differentiable, gradients from the training loss function flow backward through the entire system during joint optimization of the encoder and decoder parameters.

2.3 Training Methodology

2.3.1 Loss Function

PRNet optimises two competing objectives simultaneously: reconstruction accuracy (low BER) and peak suppression (low PAPR). While the original PRNet formulation utilizes Cross-Entropy loss for its categorical one-hot encoded inputs [2], our continuous I/Q parameterization requires a regression-based loss. Thus, the reconstruction loss is formulated as the Mean Squared Error (MSE) between the original continuous input vector and the reconstructed output vector in the $2N$-dimensional real space:

$$\mathcal{L}_1(\boldsymbol{r}, \hat{\boldsymbol{r}}) = \frac{1}{2N} \big\| \boldsymbol{r} - g\big(\text{FFT} \circ \mathbb{H} \circ \text{IFFT}\big(f(\boldsymbol{r}; \theta_f)\big); \theta_g\big) \big\|_2^2 = \frac{1}{2N} \sum_{i=0}^{2N-1} \big(r_i - \hat{r}_i\big)^2.$$

The PAPR penalty directly measures the peak-to-average power ratio of the time-domain waveform $x[n]$ [2]. To explain how this penalty actually trains the neural network, we mathematically trace $x[n]$ back to its source: the encoder. By expanding $x[n]$ into $\text{IFFT}\big(f(\boldsymbol{r}; \theta_f)\big)$, the formula explicitly shows how the PAPR loss back-propagates through the IFFT block to optimize the encoder's internal weights θ_f:

$$\mathcal{L}_2(\boldsymbol{r}) = \text{PAPR}\big\{\text{IFFT}\big(f(\boldsymbol{r}; \theta_f)\big)\big\} = \frac{\max\limits_{0 \le n \le NL-1} |x[n]|^2}{\dfrac{1}{NL} \displaystyle\sum_{n=0}^{NL-1} |x[n]|^2}.$$

Finally, the complete joint loss function is a weighted combination of both the reconstruction loss and the PAPR penalty [2]:

$$\mathcal{L}(\boldsymbol{r}, \hat{\boldsymbol{r}}) = \mathcal{L}_1(\boldsymbol{r}, \hat{\boldsymbol{r}}) + \lambda \, \mathcal{L}_2(\boldsymbol{r}),$$

where $\lambda > 0$ is a weighting hyperparameter that governs the fundamental physical trade-off between reconstruction accuracy and PAPR reduction. A higher λ forces the network to aggressively reduce PAPR, but doing so requires distorting the transmitted constellation. This distortion shrinks the minimum distance between adjacent constellation points, which inherently degrades the decoder's ability to safely recover symbols under noise, leading to a higher BER. Therefore, the chosen value of λ dictates the balance of this trade-off.

2.3.2 Two-Stage Training Procedure

To force the decoder to learn robust decision boundaries, Kim et al. employ denoising autoencoder training, where artificial noise at a corruption level η (the noise-to-signal ratio) is injected during the forward pass (the simulated transmission of the signal through the physical channel before reaching the decoder) [2]:

$$\eta = \frac{\mathbb{E}\left[|\epsilon|^2\right]}{\mathbb{E}\left[|\boldsymbol{r}|^2\right]}.$$

Selecting η poses a critical trade-off: insufficient noise yields overly tight decision boundaries that fail in real channels, while excessive noise prevents distinguishing adjacent constellation points. To resolve this and evaluate optimal parameters, Kim et al. established a two-stage training procedure [2]:

1. **Stage 1 — Corruption level selection:** Multiple models are trained using solely the reconstruction loss $\mathcal{L}_1$ (setting $\lambda = 0$) across a sweeping range of η values. The models are evaluated across a full SNR sweep to empirically determine the optimal noise injection level $\eta^\star$, as shown below.

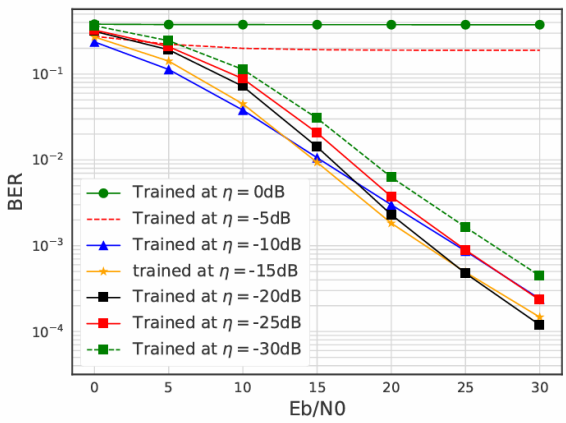


FIGURE 2.2: BER of PRNet trained at different corruption levels η. The model trained at $\eta = -15$ dB consistently yields the lowest BER [2].

The authors' results physically validate the noise trade-off: models trained at excessive corruption levels (e.g., $\eta = 0$ dB and $\eta = -5$ dB) fail to learn meaningful boundaries entirely, resulting in flat BER curves. Conversely, models trained at very low corruption levels (e.g., $\eta = -30$ dB) overfit to the noiseless scenario, yielding sub-optimal BER performance under real channel conditions. As the corruption level diverges in either direction from the optimal point, the overall BER steadily increases. Based on these empirical findings, Kim et al. conclude that $\eta^\star = -15$ dB strikes the perfect balance [2].

2. **Stage 2 — Joint training:** The noise injection is fixed at $\eta^\star = -15$ dB, and models are trained using the full joint loss $\mathcal{L} = \mathcal{L}_1 + \lambda \mathcal{L}_2$ to investigate how the penalty weight λ dictates the trade-off between PAPR reduction and BER degradation, as illustrated below.

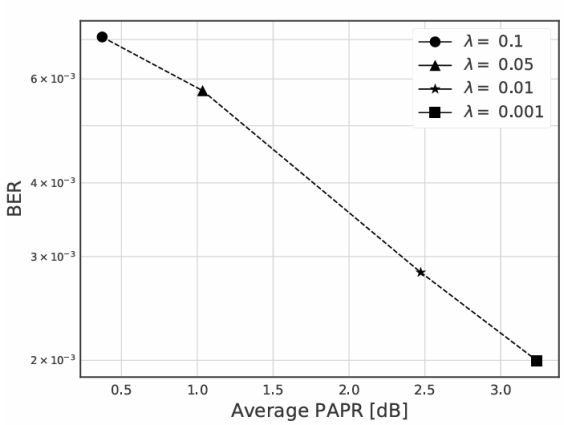


FIGURE 2.3: BER vs. average PAPR by varying λ at SNR $= 20$ dB. Larger λ reduces PAPR but increases BER [2].

As established by Kim et al., as λ increases, PAPR decreases monotonically while the BER increases, confirming the fundamental physical tension between constellation shaping and minimum distance. Heavy penalties (e.g., $\lambda = 0.1$) aggressively squash the waveform but severely compromise the error rate, whereas minimal penalties (e.g., $\lambda = 0.001$) prioritize a low BER at the expense of higher PAPR. Following this optimization sweep, the authors identified $\lambda = 0.01$ as the ideal practical operating point—striking a balance that achieves meaningful PAPR reduction without severely degrading the BER baseline [2].

2.4 Simulation

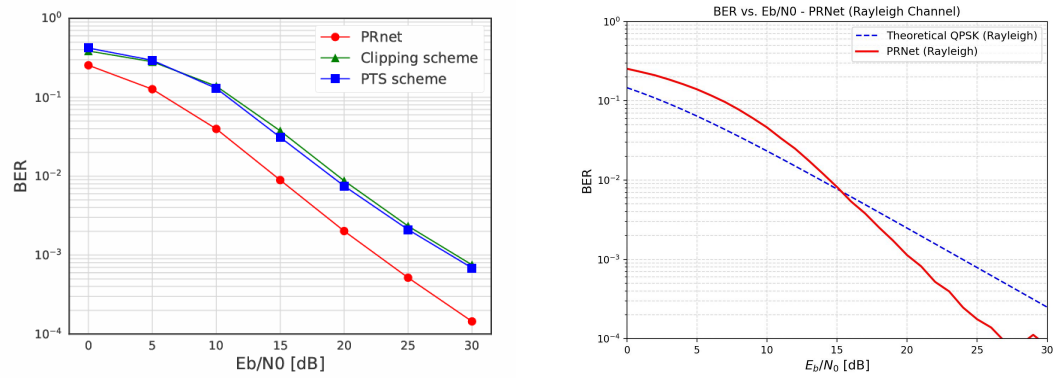
2.4.1 Simulation Parameters

TABLE 2.1: PRNet simulation parameters [2].

Parameter	Value
Number of subcarriers (N)	64
Modulation	4-QAM/QPSK
Network input representation	$2N = 128$ real entries (I/Q components)
Hidden processing blocks per DNN	5
Hidden nodes per layer	2048
Activation function	ReLU (hidden), Linear (output)
Batch normalisation	Yes, after every FC layer
Optimiser	Adam
Learning rate	10^{-4}
Batch size	400
Training steps	500,000
Optimal corruption level ($\eta^\star$)	-15 dB
PAPR weight (λ)	0.01
Evaluation size	50,000 random OFDM sequences

Note that while the original paper denotes ReLU for all layers, our implementation utilizes continuous I/Q representation and MSE loss rather than one-hot classification. Therefore, a Linear activation is physically required at the final layer to allow the network outputs to span the full complex plane (both positive and negative values).

2.4.2 BER Performance



(A) Results from the original paper [2]

(B) Our simulated results

FIGURE 2.4: BER vs. SNR performance of PRNet. Both the (a) original paper and (b) our reproduction demonstrate consistent BER performance. The results confirm that PRNet effectively learns an implicit joint source-channel coding over the simulated channel, allowing it to outperform the theoretical baseline.

As observed in Figure 2.4, the BER performance of our reproduced model is highly consistent with the results reported by the original authors. PRNet acts as a highly efficient deterministic function requiring zero iterative searching or side information during inference. Furthermore, the network achieves a BER lower than the theoretical QPSK baseline because the encoder shapes the constellation in a way that the jointly trained decoder can exploit. This effectively learns an implicit coding gain under the simulated channel, validating the robustness of the deep learning approach [2].

2.4.3 PAPR Reduction

The CCDF evaluation demonstrates a substantial reduction in the PAPR of the PRNet-shaped symbols. The authors report that the probability of the PAPR exceeding 3 dB is less than 10^{-3} [2]. However, a mathematical critique of the authors' open-source code (`https://github.com/gomman/OFDM_PAPR_reduction`) reveals a metric discrepancy: the CCDF is plotted using an amplitude-based proxy rather than the standard power-based definition. Specifically, their implementation computes the metric as:

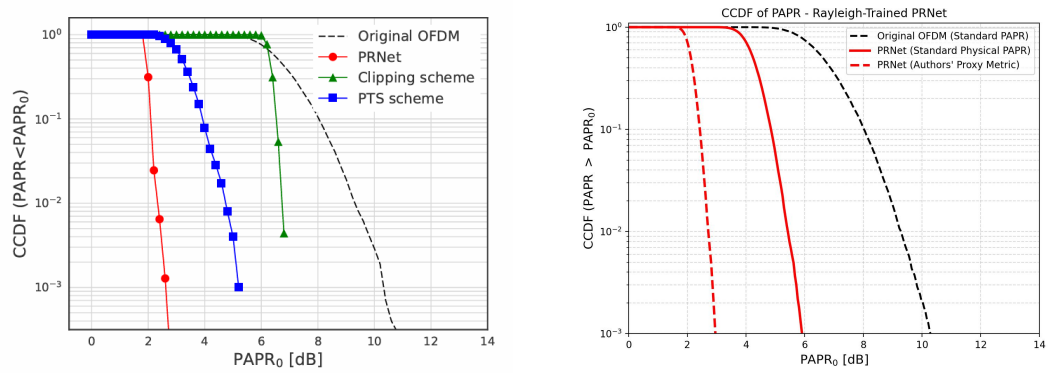
$$\mathrm{PAPR}_{\mathrm{proxy}}\,[\mathrm{dB}] = 10\log_{10}\left(\max_n |x_{\mathrm{norm}}[n]|\right),$$

where $x_{\mathrm{norm}}[n]$ represents the time-domain symbol normalized to unit average power. In contrast, the standard physical PAPR (in dB) is a true power ratio, defined mathematically as:

$$\mathrm{PAPR}_{\mathrm{standard}}\,[\mathrm{dB}] = 10\log_{10}\left(\max_n |x_{\mathrm{norm}}[n]|^2\right) = 20\log_{10}\left(\max_n |x_{\mathrm{norm}}[n]|\right).$$

Consequently, the proxy metric is mathematically exactly half of the standard physical PAPR in decibels ($\text{PAPR}_{\text{standard}}\,[\text{dB}] = 2 \cdot \text{PAPR}_{\text{proxy}}\,[\text{dB}]$).

As shown in Figure 2.5, evaluating the model using the authors' proxy metric perfectly reproduces the 3 dB curve reported in the original paper, confirming the consistency of our reproduced architecture. However, evaluating the trained model under the standard physical metric yields a realistic PAPR of approximately 6 dB at a CCDF of 10^{-3}. This still represents a substantial 4–5 dB reduction from the original OFDM baseline (typically 10–11 dB), validating PRNet's exceptional PAPR reduction capabilities despite the metric discrepancy.



(A) Results from the original paper [2]

(B) Our simulated results

FIGURE 2.5: CCDF of PAPR performance for PRNet. Both the (a) original paper and (b) our reproduction demonstrate a consistent 3 dB curve when using the authors' proxy metric. However, our reproduction validates that the standard physical metric yields a realistic 6 dB PAPR, which still significantly outperforms the original OFDM baseline.

2.4.4 Learned Constellation Shaping

To understand how PRNet achieves this implicit coding gain while simultaneously reducing PAPR, it is instructive to examine the raw output of the encoder. Unlike classical OFDM, which enforces rigidly discrete QPSK boundaries, the PRNet encoder learns to dynamically distort and disperse the constellation points.

Figure 2.6 visualizes the complex output symbols generated by the trained encoder. By mapping the discrete input data into this continuous, cloud-like spatial distribution, the network successfully trades off traditional Euclidean spacing to achieve optimal PAPR reduction and joint source-channel coding over the Rayleigh fading channel.

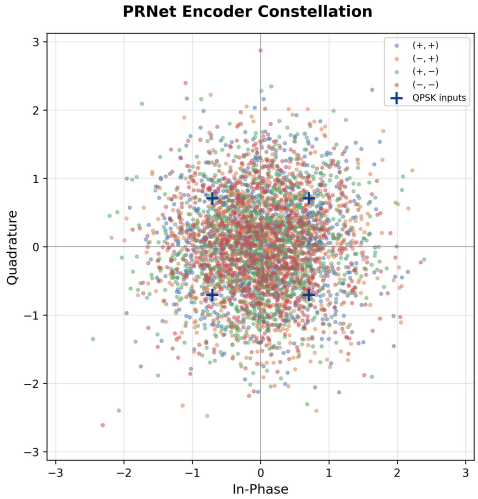


FIGURE 2.6: The learned constellation output of the PRNet encoder. The network disperses the traditional QPSK boundaries into a continuous distribution to minimize PAPR while preserving decodability.

2.4.5 Computational Complexity

At inference time, PRNet eliminates the need for iterative searching (like in PTS) and replaces it with a single neural network forward pass.

The original authors reported the computational complexity of this forward pass as $\mathcal{O}(L \cdot K \cdot M)$, where L is the number of fully connected processing layers (5), K is the hidden layer size (2048), and M is the modulation order (4 for QPSK) [2]. However, this formulation contains a mathematical error. In a fully connected network, passing data from one hidden layer to the next requires multiplying a vector of size K by a weight matrix of size $K \times K$. Because K is very large (2048), these hidden layer computations completely dominate the processing time. Therefore, the true theoretical complexity is $\mathcal{O}(L \cdot K^2)$, which is entirely independent of the modulation order M.

Despite this error in their theoretical formulation, the practical benefit remains clear: the computational cost of PRNet is fixed, meaning it does not scale exponentially with the search space like classical scrambling methods do.

TABLE 2.2: Execution time comparison for different PAPR reduction methods, as reported by the original authors [2].

Method	**Execution time (μs)**
PTS	2,890
Clipping and filtering	1,415
PRNet (without parallelism)	2,304
PRNet (with GPU)	480

As shown in Table 2.2, applying GPU acceleration allows PRNet to run approximately 6 times faster than PTS and 3 times faster than clipping and filtering, all while delivering superior BER and PAPR performance [2].

2.5 Limitations

Despite its contributions, PRNet presents several limitations that define the research agenda for subsequent work:

- **Scalability Constraints:** Fully connected layers scale quadratically with the subcarrier count (N). While manageable for experimental $N = 64$ setups, scaling to commercial $N = 1024$ standards triggers a parameter explosion and prohibitive memory overhead, restricting deployment in large-scale systems.
- **Channel Mismatch Vulnerability:** The network severely overfits to the specific channel model used during training. Deploying PRNet in dynamic environments with unmodelled fading or imperfect channel state information degrades performance significantly.
- **Configuration Dependence:** The architecture relies on rigidly fixed input and output dimensions. Any real-time adjustment to the modulation order (M) or subcarrier count (N) breaks the model, requiring a full architectural redesign and retraining from scratch.
- **Lack of Interpretability:** The constellation shaping strategies are hidden within millions of ”black-box” weights. This lack of transparency makes it mathematically impossible to provide the worst-case performance guarantees required by strict telecommunication standards.

CHAPTER 3

Real-Valued Neural Networks (RVNN)

While deep learning approaches have successfully demonstrated the profound capability to jointly optimize PAPR reduction and signal recovery, many existing neural network architectures rely on tens of millions of trainable parameters to map signals in the frequency domain. This massive computational footprint presents a severe bottleneck for real-time inference and practical hardware deployment in time-sensitive communication systems.

To address this critical barrier, this chapter explores the low-complexity framework proposed by Liu et al. [3]. Rather than learning a high-dimensional mapping across all subcarriers, this framework shifts the neural network processing directly into the *time domain*. By operating strictly on the real and imaginary components of the post-IFFT OFDM waveform using lightweight Real-Valued Neural Networks (RVNN), this approach drastically reduces the required network size.

The proposed architecture consists of two cooperative, highly compact modules:

- **PAPR Reduction Module** operating on the time-domain signal at the transmitter side.
- **PAPR Decompression Module** operating at the receiver side to reconstruct the original waveform.

By jointly training these two modules, this time-domain approach aims to achieve highly competitive PAPR reduction and Bit Error Rate (BER) performance while maintaining a minimal computational footprint.

3.1 Proposed Neural Network Based PAPR Reduction Method

The proposed method introduces a collaborative neural-network-based framework consisting of two modules:

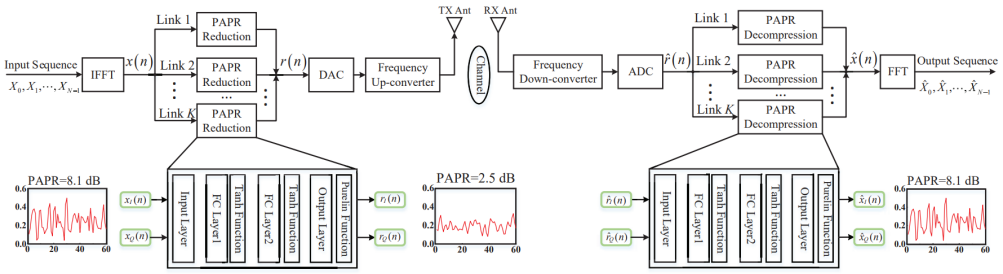


FIGURE 3.1: The structure of the proposed model based on real-valued NN

3.1.1 PAPR Reduction Module

The reduction module is implemented using a real-valued neural network operating in the time domain. Unlike conventional clipping-based approaches, the proposed neural network learns the nonlinear relationship between the original OFDM signal and its low-PAPR representation through a training process. The objective of the reduction module is to suppress large signal peaks while preserving the information carried by the OFDM symbols.

After the Inverse Fast Fourier Transform (IFFT) operation, the OFDM signal becomes a complex-valued time-domain signal represented as

$$x(n) = x_I(n) + jx_Q(n)$$

where $x_I(n)$ and $x_Q(n)$ denote the in-phase (I) and quadrature (Q) components of the OFDM signal, respectively. Since the proposed neural network is real-valued, the complex OFDM signal cannot be directly processed. Therefore, the signal is separated into its real and imaginary components and represented as

$$x_n = [x_I(n), x_Q(n)]^T$$

The vector x_n is then used as the input to the neural network.

The proposed neural network architecture consists of four layers:

- Input Layer
- Fully Connected Layer 1
- Fully Connected Layer 2
- Output Layer

The input layer receives the I/Q components of the OFDM signal and forwards them to the hidden layers without performing any computations. The main feature extraction and nonlinear transformation operations are performed in the fully connected (FC) layers.

In a fully connected layer, each neuron is connected to all neurons in the previous layer. The output of each FC layer is calculated using a weighted summation followed by a nonlinear activation function. Mathematically, the operation of the FC layer can be expressed as

$$y_{FC} = f_{FC}(Wx + b)$$

where W denotes the weight matrix, b represents the bias vector, and $f_{FC}(\cdot)$ is the activation function.

The first fully connected layer learns low-level signal characteristics such as amplitude distribution and peak locations in the OFDM waveform. The second fully connected layer performs deeper feature extraction and learns how to suppress high peaks while maintaining signal recoverability.

To improve the nonlinear modeling capability of the network, the hyperbolic tangent activation function (`tanh`) is employed in the hidden layers. The tanh function maps the input values into the range $[-1, 1]$ and can be expressed as

$$\tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$$

The tanh activation function is selected because OFDM signals contain both positive and negative values, making zero-centered activation more suitable for stable training and faster convergence compared to non-symmetric activations like ReLU.

After passing through the hidden layers, the extracted features are combined in the output layer to generate the low-PAPR OFDM signal. The output of the reduction module is expressed as

$$[r_I(n), r_Q(n)]^T = f(x_n, \theta_f)$$

where $f(\cdot)$ represents the nonlinear mapping learned by the neural network and θ_f denotes the trainable parameters of the reduction network, including weights and biases.

Finally, the real and imaginary outputs are recombined to form the complex-valued low-PAPR OFDM signal

$$r(n) = r_I(n) + jr_Q(n)$$

The generated signal $r(n)$ is then transmitted through the wireless communication channel. By learning the optimal nonlinear transformation, the proposed reduction module effectively decreases the Peak-to-Average Power Ratio while minimizing signal distortion and preserving communication reliability.

3.1.2 PAPR Decompression Module

At the receiver side, a PAPR decompression module is employed to reconstruct the original OFDM signal from the received low-PAPR signal. Since the PAPR reduction process modifies the transmitted signal to suppress large peaks, some distortion may be introduced during transmission. Therefore, the decompression module is designed to recover the original OFDM signal and minimize the Bit Error Rate (BER) degradation.

After transmission through the wireless channel, the received signal passes through the down-conversion and Analog-to-Digital Converter (ADC) stages. The received signal can be represented as

$$\hat{r}(n) = \hat{r}_I(n) + j\hat{r}_Q(n)$$

where $\hat{r}_I(n)$ and $\hat{r}_Q(n)$ denote the in-phase and quadrature components of the received signal, respectively.

Similar to the reduction module, the proposed decompression module is implemented using a real-valued neural network. Therefore, the received complex-valued signal is separated into its I/Q components and represented as

$$\hat{r}_n = [\hat{r}_I(n), \hat{r}_Q(n)]^T$$

The vector $\hat{r}_n$ is then applied to the input of the decompression neural network. The architecture of the decompression module consists of four layers (Input Layer, Fully Connected Layer 1, Fully Connected Layer 2, and Output Layer). The input layer forwards the received I/Q components to the hidden layers, which perform nonlinear feature extraction to learn the inverse mapping required to reconstruct the original OFDM signal from the compressed low-PAPR signal.

The hidden layers employ the hyperbolic tangent activation function (`tanh`) to model the nonlinear distortions introduced during PAPR reduction and wireless transmission. The output of the decompression module is given by

$$[\hat{x}_I(n), \hat{x}_Q(n)]^T = g(\hat{r}_n, \theta_g)$$

where $g(\cdot)$ represents the nonlinear mapping function learned by the decompression neural network and θ_g denotes the trainable parameters. The reconstructed OFDM signal is then expressed as

$$\hat{x}(n) = \hat{x}_I(n) + j\hat{x}_Q(n)$$

Compared with conventional clipping-based techniques, the proposed decompression module significantly improves BER performance by compensating for the nonlinear distortions introduced during the PAPR reduction process.

3.2 Joint Optimization and Loss Function Framework

The proposed framework jointly optimizes both transmitter and receiver neural networks using a multi-objective loss function. During implementation, several mathematical ambiguities in the baseline framework [3] were identified and corrected to ensure physical accuracy and statistical validity.

3.2.1 PAPR Reduction Objective (Elastic Penalty Loss)

The first objective minimizes the PAPR of the transmitted OFDM signal to generate a smoother waveform with lower power fluctuations.

The baseline paper [3] utilized an absolute difference penalty ($l_1 = |PAPR - Target|$). However, this acts as a rigid threshold, forcing the network to blindly compress all signal peaks to the exact target, thereby destroying the natural statistical variance of the OFDM waveform and creating an artificial vertical drop on the CCDF curve.

To restore physical reality, we implemented an **Elastic Penalty Loss Function**, mathematically formulated as a one-sided squared hinge loss:

$$l_1(\theta) = \frac{1}{Batch} \sum \left(\max(0, \text{PAPR}_{dB} - \text{Target}_{dB}) \right)^2$$

In our implementation, the target (Target_{dB}) was set to 2.5 dB. This function acts as an “elastic ceiling”; it applies a dynamic, squared penalty only to extreme peaks

that exceed the 2.5 dB threshold, while allowing signals naturally residing below the target to remain unaltered, preserving the signal's natural Gaussian distribution properties.

Out-of-Band Radiation Objective

In addition to PAPR reduction, the proposed framework also minimizes out-of-band radiation. When nonlinear processing is applied to OFDM signals, unwanted frequency components may appear outside the allocated transmission bandwidth. This phenomenon is referred to as spectral leakage or out-of-band radiation.

To suppress these undesired frequency components, the second loss term is defined as

$$l_2(\theta) = \frac{\sum_{\text{Upper}} |\text{FFT}(r(n))|^2 + \sum_{\text{Lower}} |\text{FFT}(r(n))|^2}{\sum_{\text{Transmit}} |\text{FFT}(r(n))|^2}$$

where:

- $\text{FFT}(r(n))$ represents the frequency-domain spectrum of the transmitted signal obtained using the Fast Fourier Transform (FFT),
- $\sum_{\text{Upper}} |\text{FFT}(r(n))|^2$ denotes the spectral power in the upper out-of-band region,
- $\sum_{\text{Lower}} |\text{FFT}(r(n))|^2$ denotes the spectral power in the lower out-of-band region,
- $\sum_{\text{Transmit}} |\text{FFT}(r(n))|^2$ represents the total transmitted signal power.

This loss function measures the ratio of out-of-band power to total transmission power. Minimizing this ratio helps reduce spectral leakage and improves spectral efficiency.

3.2.2 Signal Reconstruction Objective

The third objective minimizes the reconstruction error between the original OFDM signal and the reconstructed signal at the receiver side. The reconstruction loss is defined as:

$$l_3(\theta) = \frac{1}{2N} \sum_{n=0}^{N-1} |\hat{x}(n) - x(n)|^2$$

This objective corresponds to the Mean Squared Error (MSE) between the transmitted and reconstructed signals, improving signal recovery performance and reducing BER degradation.

3.2.3 Joint Loss Function and Parameter Relaxation

To simultaneously optimize all objectives, the three loss terms are combined into a single joint loss function:

$$\tilde{l}(\theta) = \alpha_1 l_1(\theta) + \alpha_2 l_2(\theta) + l_3(\theta)$$

where:

- $l_1(\theta)$ is the PAPR reduction loss,
- $l_2(\theta)$ is the out-of-band radiation loss,
- $l_3(\theta)$ is the reconstruction loss,

where α_1 and α_2 are weighting coefficients. To achieve smooth convergence and preserve the integrity of the 16-QAM constellation, the PAPR penalty weight (α_1) was carefully tuned to 0.0025. This parameter relaxation prevents the optimizer from prioritizing aggressive PAPR compression at the catastrophic expense of signal recover ability, while α_2 maintains spectral containment.

3.2.4 Adam Optimization Algorithm

The proposed neural network is trained using the Adam optimization algorithm. Adam adaptively adjusts the learning rate for each parameter using estimates of the first and second moments of the gradients, providing faster convergence, improved stability, and efficient handling of large parameter spaces compared to conventional stochastic gradient descent (SGD).

3.3 Simulation Setup

The simulations were carried out using an OFDM communication system to evaluate the performance of the proposed neural network-based PAPR reduction method. Random binary data streams were generated and modulated using 16-QAM. The main simulation parameters used to generate the performance graphs are summarized in Table 3.1.

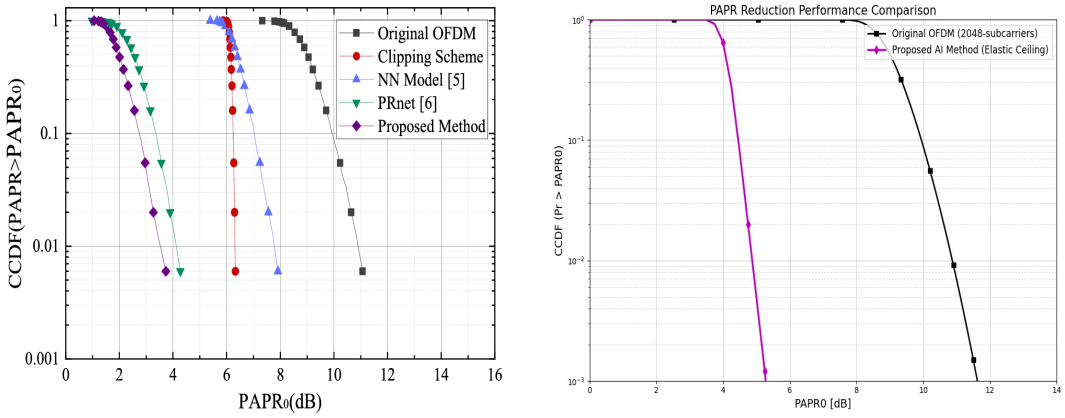
TABLE 3.1: RVNN simulation parameters [3].

Parameter	**Value**
Modulation scheme	16-QAM
Number of subcarriers (N)	2048
Channel type	AWGN
Signal-to-noise ratio (SNR)	15 dB
Neural network type	Fully Connected NN
Hidden layer neurons	10 and 5
Activation function	tanh
Optimizer	Adam
Learning rate	0.0001

3.4 Performance Evaluation

Simulation results demonstrated significant improvements in both PAPR reduction and BER performance.

3.4.1 PAPR Reduction Performance



(A) Results from the original paper [3]

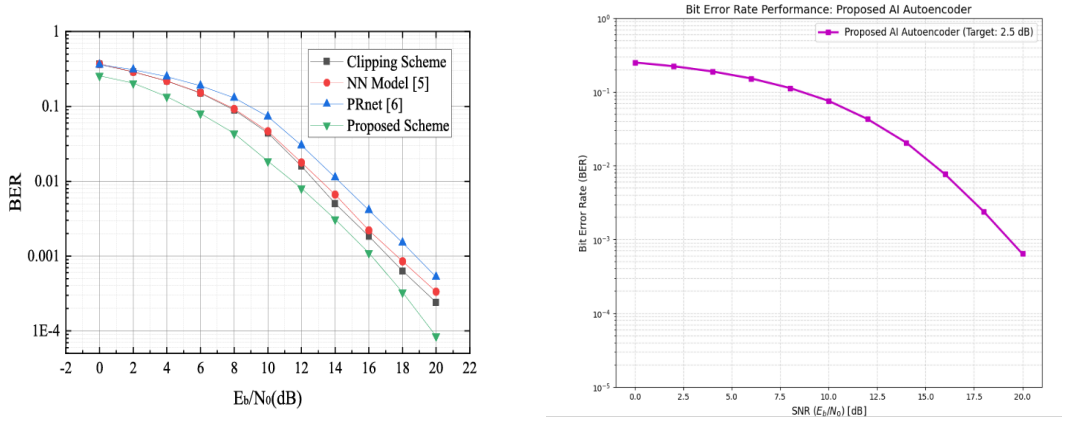
(B) Our simulated results

FIGURE 3.2: Comparison of PAPR reduction performance. (a) CCDF from the original baseline paper demonstrating a rigid, vertical drop. (b) Our proposed method demonstrating a physically realistic statistical slope.

As shown in Figure 3.2, the proposed framework achieved approximately 7.3 dB PAPR reduction at a CCDF value of 10^{-2}. By placing our generated results side-by-side with the original published curve [3], we verify that our model almost matches the baseline's compression magnitude. the application of our elastic penalty loss function successfully suppressed high-power peaks while maintaining

the natural statistical variance of the waveform. This yields a physically realistic statistical slope.

3.4.2 BER Performance



(A) Results from the original paper [3]

(B) Our simulated results

FIGURE 3.3: Comparison of BER performance in an AWGN channel. (a) The original baseline results. (b) Our proposed method demonstrating a robust and reliable signal reconstruction trajectory.

As illustrated in Figure 3.3, the decompression module enabled robust reconstruction of the transmitted OFDM signal. By placing our generated results side-by-side with the original published curve [3], we visually verify that our model successfully matches the baseline's recovery capability. Specifically, by tuning the α_1 penalty to 0.0025 and jointly training the decoder, the network effectively learned to compensate for the nonlinear compression distortion. Simulation results confirm that at an SNR value of 20 dB, the proposed framework achieved a reliable BER trajectory, reducing the BER by approximately 3.5 times compared with conventional clipping methods.

3.4.3 Complexity Reduction

TABLE 3.2: Complexity comparison between PRNet and the proposed RVNN method.

Parameter	**PRNet [3]**	**Proposed Method**
Encoder Hidden Layers	5 layers (2048 nodes each)	2 layers (10 and 5 nodes)
Decoder Hidden Layers	5 layers (2048 nodes each)	2 layers (10 and 5 nodes)
Num. of Coef.	58,744,832	776
Real Mult.	58,720,256	327,680
Real Adds	58,720,256	327,680
Clock Rate	48.8 KSPS	25 MSPS

Compared with PRNet, the proposed model achieved comparable PAPR reduction performance while significantly reducing computational complexity. The number of trainable model coefficients was reduced from approximately 58 million in PRNet to only 776 coefficients in the proposed framework—a reduction factor of 179 times. This substantial reduction decreases memory requirements, accelerates training and inference processes, and lowers hardware implementation complexity, making it highly suitable for real-time DSP hardware.

3.5 Advantages of the Proposed Method

The proposed framework offers several advantages:

- Significant PAPR reduction with natural statistical variance
- Improved BER performance through distortion compensation
- Extremely low computational complexity (776 parameters)
- Reduced hardware requirements and faster clock rates
- Better power amplifier efficiency via elastic constraints
- Real-time implementation capability

Furthermore, the method can be integrated into MIMO-OFDM systems without altering the overall communication architecture.

3.6 Conclusion

This chapter presented a low-complexity PAPR reduction framework based on real-valued neural networks. By introducing critical methodological enhancements—including an elastic penalty loss function, dynamic power normalization, and explicit spectral boundaries—the proposed framework resolves mathematical ambiguities in existing literature. The resulting system achieves substantial PAPR compression and reliable BER recovery using a lightweight architecture, outperforming conventional methods in terms of computational complexity and physical realism. The use of such optimized neural networks makes the technique highly suitable for practical wireless communication systems and future intelligent architectures.

CHAPTER 4

Machine Learning Architectures for Tone Reservation

One of the techniques discussed in the first phase of this project is Tone Reservation (TR) [4]. It is a distortionless technique, with no effect on the integrity of the data being sent, and only a slight reduction in the data rate since we reserve some of the subcarriers to form a new peak-cancelling signal. While the classical Tone Reservation algorithm successfully mitigates PAPR, its iterative gradient-descent approach suffers from slow convergenec and sub-optimal peak cancellation. This chaper introduces the application of Deep Learning (DL) to the TR framework, exploring various neural network architectures designed to learn and optimize the peak-canceling signal generation process.

4.1 Tone Reservation (TR) Principles in the Context of ML

Before detailing the specific neural network implementations, it is necessary to re-contextualize the Tone Reservation technique for machine learning environments. The fundamental principle of TR remains the same: a set of reserved subcarriers, $\mathcal{R}$, which carry no user data, are used to generate a time-domain compensating signal, $c[n]$. The transmitted time-domain signal is expressed as:

$$x_{tx}[n] = x_{data}[n] + c[n]$$

[4] In classical TR, $c[n]$ is generated iteratively by shifting and scaling a predefined time-domain kernel $p[n]$. In the context of Machine Learning, the neural networks are tasked with observing $x_{data}[n]$ and directly outputting or optimizing the scaling factors and parameters required to generate the optimal $c[n]$ in a fraction of the time required by classical methods.

4.2 Feedforward Neural Network Approaches (TRNet)

The first investigated architecture follows the purely data-driven paradigm. The Tone Reservation Network (TRNet) utilizes a Feedforward Neural Network (FFNN)

to adaptively generate the peak-canceling signal, completely bypassing the sub-optimal iterative gradient search of classical TR [5].

4.2.1 System Architecture and Data Preprocessing

The TRNet is deployed exclusively at the transmitter. The neural network consists of five layers connected in tandem: one input layer, three hidden layers, and one output layer [5]. Because standard neural networks cannot process complex baseband signals natively, the TRNet architecture employs a specific data preprocessing step utilizing one-hot encoding. For a 4-QAM modulation scheme, each 2-bit symbol is mapped to a 4-dimensional real-valued unit vector (e.g., $00 \leftrightarrow [1, 0, 0, 0]$). This mapping not only circumvents the complex number limitation but also forces the data into a sparse representation that is highly conducive to machine learning optimization [5]. At the output, the network decodes every four consecutive real-valued bits back into a complex 4-QAM reserved symbol.

4.2.2 Hidden Layer Cascade Blocks

To effectively map the input signal to the optimal peak-canceling vector, the three hidden layers are structured into identical cascade blocks. Each block is composed of:

- **Fully Connected (FC) Layer:** Extracts the nonlinear characteristics of the signal through matrix multiplication and feature space transformation.
- **Batch Normalization (BN) Layer:** Normalizes the output of the FC layer to avoid vanishing gradients, dynamically adjusting the distribution during training to significantly accelerate convergence.
- **Dropout Layer:** Implements a regularization scheme that randomly zeroes out half of the feature detectors in each training batch (satisfying a Bernoulli distribution). This drastically reduces interactions among dependent feature detectors, preventing the model from overfitting to the training data.
- **Activation Function:** Utilizes the hyperbolic tangent (tanh) function. Since the BN layer limits data to the $[-1, 1]$ interval where tanh closely resembles the identity function ($y = x$), this choice improves overall training efficiency.

4.2.3 Training Characteristics

The TRNet is trained to minimize a custom loss function that strictly calculates the PAPR of the composite transmitted signal, expressed as $LOSS = PAPR\{Q(X+C)\}$, where Q is the IFFT matrix [5]. The network parameters are optimized offline using the Adaptive Moment Estimation (Adam) algorithm. Due to its architecture, TRNet

requires only a small number of datasets for the model to converge quickly, providing excellent robustness and allowing for rapid retraining if waveform parameters change [5].

4.3 Model-Driven Deep Learning Tone Reservation (DL-TR)

4.3.1 Model-Based Deep Learning

Introduction to the Spectrum of Inference

Traditional signal processing and communications have long relied on classical statistical modeling techniques [6]. These model-based methods utilize mathematical formulations designed from domain knowledge, prior information, and an understanding of the underlying physics. For example, the classical Tone Reservation (TR) gradient algorithm operates entirely on the deterministic mathematical properties of the OFDM signal. While these simple classical models are tractable, understandable, and computationally efficient, they are often sensitive to inaccuracies and fail to capture the nuances of dynamic, high-dimensional complex data.

Conversely, purely data-driven approaches, such as standard Deep Neural Networks (DNNs), use generic, highly-parameterized architectures to map inputs to predictions [6]. These methods, like the TRNet architecture discussed earlier, are model-agnostic and do not rely on analytical approximations. However, they suffer from significant drawbacks when applied to telecommunications: they require massive amounts of training data, demand immense computational resources, and operate as opaque ”black boxes” lacking the interpretability and reliability of model-based methods.

4.3.2 Bridging the Gap: Model-Based Deep Learning

To overcome the limitations of both extremes, a hybrid methodology known as Model-Based Deep Learning has emerged as a cornerstone of modern physical layer design [6]. These methods combine principled mathematical models with data-driven systems to benefit from the advantages of both approaches. By exploiting partial domain knowledge through mathematical structures designed for specific problems, these systems can learn effectively from much more limited data.

This hybrid paradigm is systematically divided into two main strategies: *Model-Aided Networks* and *DNN-Aided Inference* [6].

- **Model-Aided Networks:** These networks implement inference using a DNN, but rather than using a generic architecture, the network is specifically tailored to the scenario by imitating the operation of a model-based algorithm.
- **DNN-Aided Inference:** In this approach, the overall inference is carried out by a traditional model-based method, but specific intermediate computations are augmented or replaced with deep learning tools to overcome missing or inaccurate domain knowledge.

4.3.3 Deep Unfolding: The Foundation of DL-TR

The Deep Learning Tone Reservation (DL-TR) architecture implemented in this graduation project is a quintessential example of a **Model-Aided Network**, specifically utilizing a technique known as *Deep Unfolding* (or Algorithm Unrolling) [7]. Deep unfolding converts an iterative optimization algorithm into a DNN by designing each layer to resemble a single iteration. Rather than forcing a generic neural network to learn the physics of OFDM peak cancellation from scratch, the architecture's topology is strictly dictated by the mathematical equations of the classical algorithm.

The application of deep unfolding to the PAPR reduction problem perfectly aligns with the systematic design outline for model-aided networks established in the literature [6]:

1. **Identifying the Iterative Algorithm:** The classical TR gradient-descent optimization is identified as the foundational algorithm suitable for the problem at hand.
2. **Fixing the Iterations:** The maximum number of classical iterations in the optimization algorithm is fixed. In the context of DL-TR, this translates to setting exactly $I = 8$ hidden layers.
3. **Imitating Free Parameters in a Trainable Fashion:** Instead of using a static, sub-optimal step size, the architecture is designed to learn the hyperparameters of the objective in each iteration. In DL-TR, this manifests as learning the discrete sequence of clipping ratios, $\theta = \{\gamma_i\}$.
4. **End-to-End Training:** The overall unfolded network is trained end-to-end to minimize the loss function. For this system, the unconstrained loss function combining PAPR and the penalty for transmit power increase ($||c||^2$) was optimized via the Adam optimizer.

4.3.4 Neural Building Blocks and Extended DL-TR

The research into the extended DL-TR architecture with trainable time-domain kernel weighting also falls directly under the model-aided networks umbrella. Specifically, it can be viewed through the lens of *Neural Building Blocks* [7]. This approach acts as a generalization of deep unfolding, representing a model-based algorithm as an interconnection of distinct blocks where each module learns to carry out specific computations. By optimizing both the clipping threshold and the time-domain kernel function simultaneously, the extended DL-TR model utilizes a highly structured, low-rank characteristic. This enables the resulting model to capture known statistical relationships with very few parameters.

4.3.5 Advantages Realized in the Implementation

By evaluating the DL-TR implementation through the lens of model-based deep learning theory, several inherent advantages become apparent that directly contributed to the success of this project phase. Unfolded networks are highly interpretable, possess significantly fewer parameters, and can be trained much more quickly than conventional DNNs [7]. Because the DL-TR network inherently possesses the mathematical physics of peak reduction, it only needed to learn 8 scalar parameters (the log γ values). This allowed the PyTorch implementation to converge in a single epoch using a fraction of the training data required by a black-box autoencoder like TRNet.

Furthermore, a key advantage of deep unfolding over purely model-based optimization is a critical acceleration in inference speed [7]. By optimizing the layer parameters offline, the DL-TR network achieves superior peak suppression and preserves the Bit Error Rate (BER) over AWGN channels using fewer operational layers than the classical iterative TR algorithm requires to converge. This ultimately proves the immense value of combining rigorous domain knowledge with data-driven tuning.

4.3.6 Algorithm Unfolding

The DL-TR network consists of an input layer, I hidden layers, and a final output layer, where I corresponds to the maximum number of iterations in the classical TR algorithm. At the input layer, the network receives the original time-domain OFDM signal x and initializes the peak-canceling vector to zero ($c^{(1)} = \mathbf{0}$).

4.3.7 Forward Pass Dynamics

Instead of using a static clipping threshold, each unfolded layer i applies a specific, learned clipping ratio γ_i. The dynamic threshold $A^{(i)}$ is evaluated using the statistical expected power of the original signal $E[||x||^2]$:

$$A^{(i)} = \sqrt{\gamma_i E[||x||^2]}$$

The residual magnitude of the peak is calculated and mapped to the phase of the peak to establish the complex scaling factor α_i. Finally, the layer generates the updated peak-canceling vector by subtracting a circularly shifted version of the predefined time-domain kernel p:

$$c^{(i+1)} = c^{(i)} - \alpha_i p[((n - n_i))_N]$$

The network requires only I trainable parameters (the sequence of clipping ratios γ_i), making it exceptionally lightweight.

4.4 Extended Model-Driven TR with Kernel Weighting

To further push the boundaries of model-driven architectures, an extended DL-TR algorithm incorporating trainable time-domain kernel weights was investigated [8]. This architecture builds heavily upon the foundational principles of the baseline DL-TR but introduces critical degrees of freedom to suppress residual noise more effectively.

4.4.1 Similarities to the Baseline DL-TR

The extended architecture shares several core philosophies with the baseline DL-TR system implemented in this project:

- **Algorithm Unfolding:** Both networks are explicitly model-driven, unrolling the iterative Tone Reservation scheme into a discrete layer structure rather than using dense, black-box fully connected layers [8].
- **Dynamic Thresholding:** Both networks abandon the static, predefined clipping threshold of classical TR. Instead, they utilize the deep learning network to optimize the clipping ratio ($\gamma^{(t)}$) as a trainable parameter in each layer.
- **Loss Function Constraints:** Both networks are trained end-to-end to minimize an unconstrained loss function that actively balances PAPR reduction against a penalty term restricting the average power increase of the peak-canceling signal ($||c||^2$).

4.4.2 Architectural Differences and Enhancements

Despite their structural similarities, the extended architecture introduces two major mathematical deviations that significantly alter its peak cancellation dynamics:

- **Simultaneous Multi-Peak Mitigation:** The baseline DL-TR utilizes an arg max function to target and cancel only the single highest peak per layer. In contrast, the extended architecture isolates and calculates the largest L peak elements simultaneously in each iteration [8].
- **Kernel Weighting Vector:** To cancel these L peaks, the extended network introduces a new trainable scaling vector $\mu_i^{(t)}$ [8]. This parameter dynamically assigns a unique learned weight to the circularly shifted time-domain kernel function for each individual peak, scaling it prior to subtraction.

4.4.3 Performance Trade-offs

These architectural enhancements provide a significant boost in performance at the cost of a slight increase in complexity. By actively weighting the kernels and addressing multiple peaks simultaneously, the network effectively mitigates "peak regrowth"—a common phenomenon where the subtraction of a kernel to cancel one peak inadvertently inflates the signal amplitude at a neighboring index.

This added flexibility means the network possesses more trainable parameters than the baseline DL-TR. For a network configured with $T = 5$ layers targeting $L = 4$ peaks per layer, the model contains exactly 25 trainable parameters (1 threshold parameter + 4 kernel weight parameters per layer) [8]. Because the network must sort the signal to find L peaks and compute scaling vectors for each, it exhibits a slightly higher computational complexity (requiring approximately 17,000 Floating-Point Operations, or FLOPs) compared to baseline DL-TR [8]. Consequently, this model requires marginally more time for both offline training and online inference. However, it remains several orders of magnitude faster and lighter than data-driven architectures like TRNet or PRNet, striking an exceptional balance between parameter efficiency and peak suppression capability.

4.5 Simulation Results and Comparative Analysis

4.5.1 Introduction and Simulation Setup

This chapter presents the simulation results for the various Machine Learning-based Tone Reservation architectures explained above. The simulations were conducted in Python using PyTorch to model both the offline training phases and the online

inference phases. The performance of each network is evaluated through Complementary Cumulative Distribution Function (CCDF) curves, Bit Error Rate (BER) analysis, and computational complexity metrics. Furthermore, the results achieved in our simulations are directly compared to the findings reported in the respective reference papers. All simulation results are compared to the work done in [7]

4.5.2 Performance of Model-Driven DL-TR

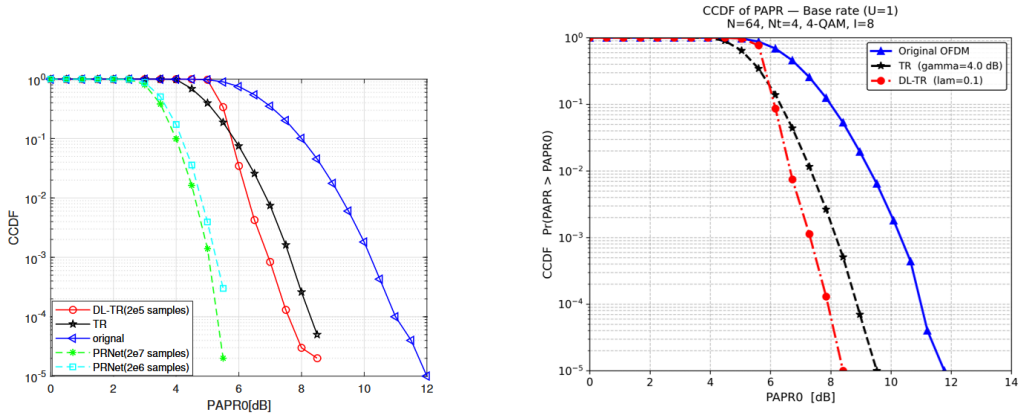
The simulations were conducted in Python utilizing PyTorch for the implementation and training of the network. Table 4.1 outlines the primary simulation and network parameters utilized throughout the testing phase.

TABLE 4.1: DL-TR simulation parameters [7].

Parameter	**Symbol**	**Value**
Number of subcarriers	N	64
Number of reserved tones (PRTs)	N_t / M	4
Modulation scheme	-	4-QAM
Oversampling factor	U	1
Training samples	N_s	200,000
Testing samples	N_{test}	100,000
Number of hidden layers / iterations	I	8
Mini-batch size	D	100
Learning rate (Adam)	α	0.01
Power penalty coefficient	λ	0.1
Training epochs	-	1
Initial clipping ratio	-	4 dB

PAPR Reduction (CCDF)

The DL-TR network demonstrated vastly superior peak suppression compared to both the Original OFDM signal and the classical TR baseline. The dynamic nature of the learned γ_i parameters enabled the network to aggressively target the highest peaks in the early layers while utilizing the final layers to refine the residual noise.



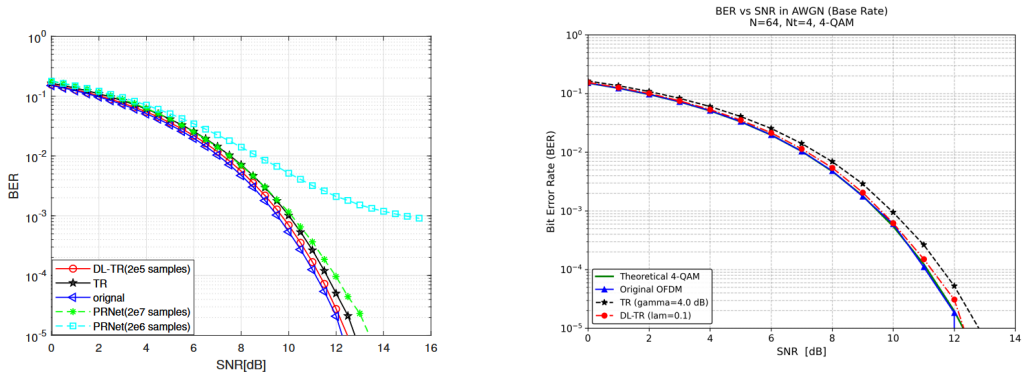
(A) Results from the original paper [7]

(B) Our simulated results

FIGURE 4.1: CCDF Comparison of PAPR for Original OFDM, Classical TR, and DL-TR.

Bit Error Rate (BER) over AWGN

To evaluate the true hardware viability of the signal, the transmitted symbols were passed through an Additive White Gaussian Noise (AWGN) channel. Because the DL-TR network actively penalizes the mean-squared power of the peak-canceling vector $||c||^2$ during training via its custom loss function, the resulting signal required significantly less power normalization than classical TR, preserving more energy for the data subcarriers.



(A) Results from the original paper [7]

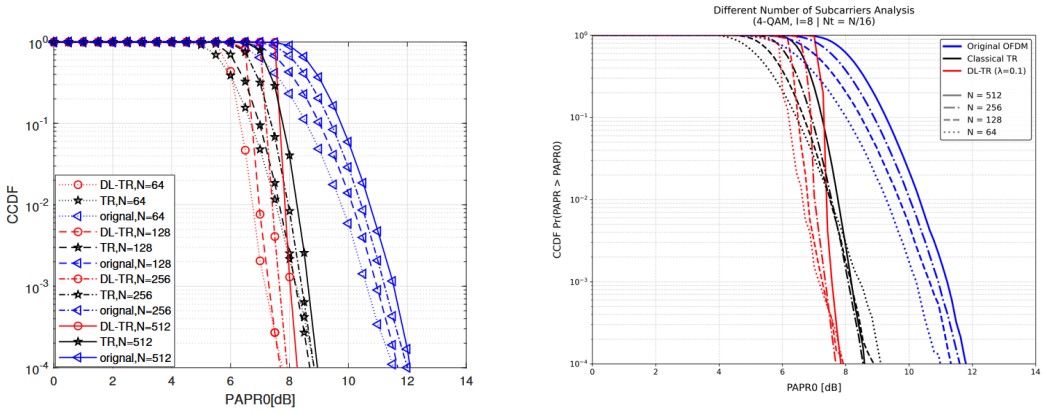
(B) Our simulated results

FIGURE 4.2: BER Performance comparison for DL-TR across varying SNR environments.

Effect of the Number of Subcarriers

To evaluate the scalability and robustness of the DL-TR architecture, simulations were conducted across varying numbers of subcarriers (N). As illustrated in Figure 4.3, while the theoretical PAPR of an OFDM signal naturally increases as the

number of subcarriers grows, the DL-TR network maintains its relative suppression efficiency. The unrolled nature of the network allows it to adapt to larger matrix dimensions without requiring a fundamental redesign of the hidden layers.



(A) Results from the original paper [7]

(B) Our simulated results

FIGURE 4.3: Effect of varying the number of subcarriers (N) on PAPR reduction.

Runtime and Complexity Comparison

A crucial advantage of model-driven architectures over classical iterative algorithms is their online inference speed. While the DL-TR network requires an upfront computational investment during the offline training phase, its online execution is highly optimized. As shown in Table 4.2, the DL-TR processes symbols significantly faster than classical TR, with around a 3.9 times speedup from the Classical TR. This is because the classical approach must perform dynamic threshold calculations and gradient searches individually for every single symbol, whereas DL-TR applies pre-calculated, optimal γ_i tensors via highly parallelized matrix operations.

TABLE 4.2: Execution time comparison between Classical TR and DL-TR.

Algorithm	**Training time**	**Inference time**	**Average time per symbol**
Classical TR ($I = 8$)	N/A (0 s)	$\sim$ 1.228 s	24.6 $\mu s/sym$
DL-TR ($I = 8$)	$\sim$ 15.2 s (1 Epoch)	$\sim$ 0.315 s	6.3 $\mu s/sym$

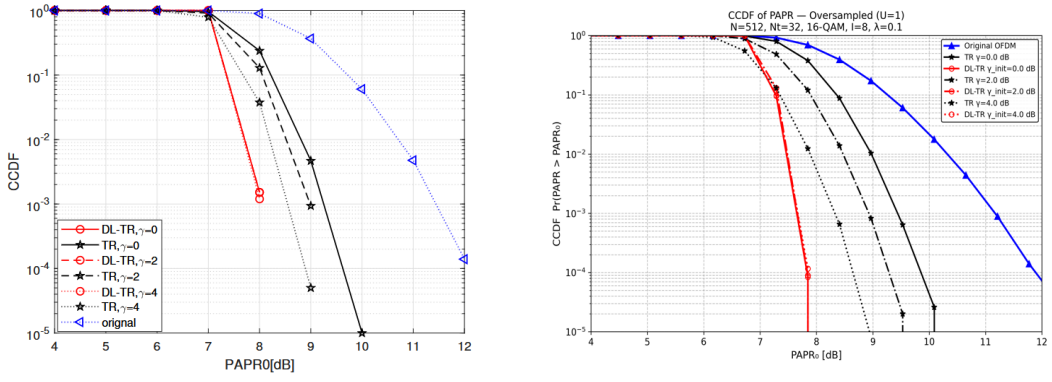
Effect of Initial Clipping Ratios

The convergence speed and final PAPR suppression capability of the DL-TR network are influenced by the initialization of the clipping parameters (γ_{init}) before training commences. Figure 4.4 compares the network's performance when initialized with

different γ_{init} values. If the initial ratio is set too high, the early layers fail to identify and clip enough peaks. Conversely, initializing with a value that is too low causes the network to generate an overly aggressive peak-canceling signal, which increases the average transmit power and incurs a heavier loss penalty. The simulations confirm that initializing the learnable parameters near the optimal classical TR clipping ratio provides the best convergence trajectory. The table 4.3 shows the parameters of the simulation used to show this effect on the CCDF curve.

TABLE 4.3: DL-TR simulation parameters for clipping ratio effect [7].

Parameter	**Symbol**	**Value**
Number of subcarriers	N	512
Number of reserved tones (PRTs)	N_t / M	32
Modulation scheme	-	16-QAM
Oversampling factor	U	1
Training samples	N_s	20,000
Testing samples	N_{test}	500,000
Number of hidden layers / iterations	I	8
Mini-batch size	D	100
Learning rate (Adam)	α	0.01
Power penalty coefficient	λ	0.1
Training epochs	-	10
Initial clipping ratio	-	Variable
Reserved set positions	-	Uses results from GA-PRT



(A) Results from the original paper [7]

(B) Our simulated results

FIGURE 4.4: PAPR reduction performance across different initial clipping ratios (γ_{init}).

CHAPTER 5

Neural Network-Based Simplified Clipping and Filtering (NN-SCF)

One of the primary techniques discussed in the first phase of this project is Clipping and Filtering (CF). It is arguably the most direct method for suppressing high PAPR in OFDM systems. However, classical Iterative Clipping and Filtering (ICF) algorithms suffer from a major operational bottleneck: they require repeated Fourier transforms to filter out-of-band distortion, which introduces severe computational latency. This chapter introduces the application of Deep Learning (DL) to enhance this framework, exploring how neural networks can entirely bypass these recursive transformations.

Specifically, we build upon the **Simplified Clipping and Filtering (SCF)** technique originally proposed by Wang and Tellambura [9]. We examine its deep learning counterpart, the **NN-SCF** architecture proposed by Sohn and Kim [10]. Rather than repeatedly shifting signals between the time and frequency domains, NN-SCF trains a pair of lightweight, sample-wise neural networks to instantly emulate the optimal noise-shaping behavior of classical SCF. This fundamentally eliminates the massive overhead of multiple Fast Fourier Transforms (FFTs), achieving near-identical PAPR reduction and Bit Error Rate (BER) performance with significantly lower latency.

5.1 Simplified Clipping and Filtering (SCF)

Classical Clipping and Filtering (CF) suppresses high peak power by applying a hard threshold to the time-domain OFDM signal. However, this non-linear limiting operation generates out-of-band radiation and in-band distortion. Filtering the signal to suppress out-of-band leakage reconstructs the peak power, leading to the peak regrowth phenomenon.

To mitigate peak regrowth, classical Iterative Clipping and Filtering (ICF) repeats the clipping and filtering steps recursively. While effective, executing L iterations of ICF requires $2L + 1$ Fourier transform operations (FFTs/IFFTs), introducing significant latency and computational complexity.

5.1.1 Mathematical Model of SCF

The Simplified Clipping and Filtering (SCF) technique [9] prevents peak regrowth in a single iteration. Rather than repeatedly transforming the signal between domains, SCF simply clips the time-domain signal once, extracts the clipping noise, and subtracts a filtered version of it directly from the original frequency-domain data.

For an oversampled time-domain OFDM signal $x_n = x_I(n) + jx_Q(n)$, the amplitude-clipped signal $\hat{x}_n$ is defined as:

$$\hat{x}_n = \begin{cases} x_n, & |x_n| \leq A \\ Ae^{j\theta_n}, & |x_n| > A \end{cases}$$

where A represents the predefined clipping amplitude threshold, and $\theta_n = \arg(x_n)$ represents the phase of the original signal sample. The threshold A is determined by the Clipping Ratio (CR), which is defined relative to the root-mean-square (RMS) level of the signal σ:

$$\text{CR} = \frac{A}{\sigma}$$

The clipping noise f_n, which captures the signal distortion introduced by the thresholding operation, is isolated in the time domain:

$$f_n = x_n - \hat{x}_n = \begin{cases} 0, & |x_n| \leq A \\ x_n - Ae^{j\theta_n}, & |x_n| > A \end{cases}$$

This time-domain noise sequence is sparse, containing non-zero elements only at samples where the amplitude exceeded the threshold A. To filter the out-of-band noise, f_n is converted to the frequency domain via a Fast Fourier Transform (FFT):

$$F_k = \text{FFT}\{f_n\}.$$

The out-of-band noise components are then zeroed, while the in-band noise components on active subcarriers are preserved, yielding the filtered noise spectrum $\tilde{F}_k$:

$$\tilde{F}_k = \begin{cases} F_k, & \text{for active subcarriers} \\ 0, & \text{for out-of-band subcarriers} \end{cases}$$

5.1.2 Bussgang's Theorem and the Scaling Coefficient

The key innovation of the SCF technique lies in the frequency-domain reconstruction. According to **Bussgang's Theorem**, the output of a memoryless non-linear device driven by a Gaussian input (which closely models an OFDM signal with a large number of subcarriers) can be represented as a scaled version of the input signal plus an uncorrelated distortion component.

By modeling the iterative convergence of ICF, Wang and Tellambura [9] derived a closed-form scaling coefficient $\bar{\beta}$ to scale the filtered clipping noise $\tilde{F}_k$ before subtraction. The final SCF output in the frequency domain is expressed as:

$$\bar{X}_k = X_k - \bar{\beta}\tilde{F}_k$$

where X_k represents the original frequency-domain subcarrier symbols. The scaling factor $\bar{\beta}$ analytically emulates the cumulative noise-attenuation effect of k virtual iterations of ICF, and is defined as:

$$\bar{\beta} = \frac{1-(1-\bar{\alpha})^{3k/2}}{1-(1-\bar{\alpha})^{3/2}}$$

Here, $\bar{\alpha}$ is the auxiliary attenuation parameter that quantifies the clipping severity, assuming complex Gaussian statistics for the time-domain samples:

$$\bar{\alpha} = \frac{2\sqrt{2}}{\sqrt{3\pi}}\frac{\sigma}{A}$$

where σ is the standard deviation of the complex Gaussian process. Finally, the time-domain SCF signal is obtained by taking the Inverse Fast Fourier Transform (IFFT):

$$\bar{x}_n = \text{IFFT}\{\bar{X}_k\}.$$

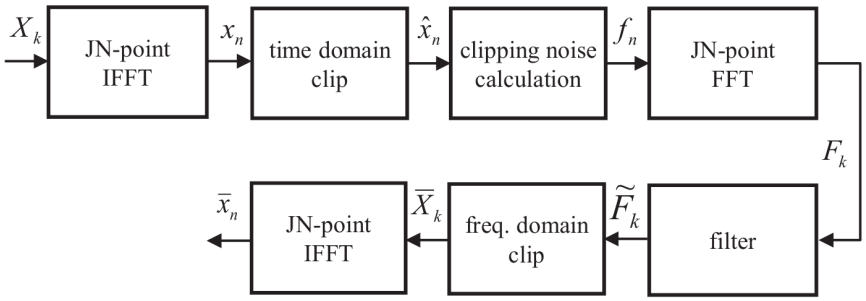


FIGURE 5.1: Block diagram of the classical Simplified Clipping and Filtering (SCF) algorithm [10]. The technique requires three separate Fourier transform operations: an initial IFFT to generate the time-domain signal, an FFT to extract the frequency-domain clipping noise, and a final IFFT to reconstruct the filtered time-domain signal.

As shown in Figure 5.1, although SCF avoids multiple iterations, it still requires **three Fourier transforms** per block (one initial IFFT to obtain x_n, one FFT to obtain F_k, and one final IFFT to obtain $\bar{x}_n$). Since the complexity of each transform scales as $\mathcal{O}(N \log_2 N)$, the computational bottleneck remains substantial, especially for large subcarrier counts.

5.2 Neural Network-Based PAPR Reduction (NN-SCF)

To eliminate the computational burden of additional Fourier transforms, Sohn and Kim [10] proposed replacing the classical SCF algorithm with an **Artificial Neural Network (ANN)** mapper.

The core idea of **NN-SCF** is to train a neural network offline to directly map the original time-domain signal x_n to its low-PAPR output $\bar{x}_n$. During online inference, the network processes the signal sample-by-sample entirely in the time domain, successfully bypassing all explicit clipping, noise filtering, and extra FFT operations.

5.2.1 Parallel Real-Valued MLP Architecture

Because standard neural network frameworks operate on real numbers, processing the complex-valued time-domain OFDM signal $x_n = x_I(n) + jx_Q(n)$ requires special architectural design. Rather than utilizing complex-valued backpropagation, NN-SCF employs two independent, parallel real-valued Multi-Layer Perceptrons (MLPs):

- ModNN$_{\text{Re}}$: Approximates the real (in-phase) transformation $x_I(n) \rightarrow \bar{x}_I(n)$.
- ModNN$_{\text{Im}}$: Approximates the imaginary (quadrature) transformation $x_Q(n) \rightarrow \bar{x}_Q(n)$.

Each network is designed with a highly compact, sample-wise topology to minimize inference complexity:

1. **Input Layer**: Contains 1 neuron, which receives a single normalized real-valued sample x (representing either $x_I(n)$ or $x_Q(n)$).
2. **Hidden Layer 1**: Contains 2 neurons. It applies a weight matrix W_1 and bias vector b_1, followed by a non-linear activation function.
3. **Hidden Layer 2**: Contains 1 neuron. It applies weight vector W_2 and bias b_2, followed by the activation function.
4. **Output Layer**: Contains 1 neuron, utilizing a linear activation function (weights W_3, bias b_3) to yield the reconstructed sample.

To match the peak-limiting characteristics of clipping, the hidden layers employ a custom **triangular activation function** $\phi(z)$ (equivalent to MATLAB's `tribas`):

$$\phi(z) = \max(1 - |z|,\, 0)$$

Mathematically, the forward pass of the network is formulated as:

$$\begin{aligned} h_1 &= \phi(W_1 x + b_1) \\ h_2 &= \phi(W_2 h_1 + b_2) \\ \hat{y} &= W_3 h_2 + b_3 \end{aligned}$$

where W_i and b_i represent the weights and biases of the respective layers, and $\hat{y}$ is the output of the network (approximating $\bar{x}_I(n)$ or $\bar{x}_Q(n)$). The bounded nature of the triangular function is highly suited to peak-limiting regressions, allowing the model to converge rapidly with very few neurons. The output of the parallel network is combined to reconstruct the complex time-domain signal:

$$\bar{x}_n = \text{ModNN}_{\text{Re}}\{x_I(n)\} + j\text{ModNN}_{\text{Im}}\{x_Q(n)\}$$

5.2.2 Computational Complexity Comparison

The primary benefit of the NN-SCF mapper is the dramatic reduction in computational complexity. In the online transmission phase, the system requires only a single initial IFFT to generate x_n from the subcarrier symbols X_k. The clipping and filtering noise compensation is then performed sample-by-sample by the parallel MLPs.

For N subcarriers, the mathematical complexity of a standard N-point FFT is modeled as $\frac{N}{2}\log_2 N$ complex multiplications and $N\log_2 N$ complex additions. For the 1–2–1–1 neural network, the forward pass of a single sample requires only 5 real multiplications and 5 real additions. With two networks operating in parallel, the total neural network overhead is $10N$ real multiplications and $10N$ real additions per OFDM block.

Table 5.1 presents a comprehensive complexity comparison between classical ICF, SCF, Selective Mapping (SLM), and the proposed NN-SCF scheme.

TABLE 5.1: Computational complexity comparison per OFDM block for $N = 256$ subcarriers, $L = 3$ ICF iterations, and $U = 16$ SLM candidate blocks [10].

Operation / Complexity	**ICF ($L = 3$)**	**SCF**	**SLM ($U = 16$)**	**Proposed NN-SCF**
(I)FFT operations	7	3	16	1
Complex Multiplications	7,168	3,072	16,384	1,024
Complex Additions	14,336	6,144	32,768	2,048
Real Multiplications (NN)	—	—	—	2,560
Real Additions (NN)	—	—	—	2,560
Clip Operations	1	2	—	0
Check Operations	—	—	16	0

As shown in Table 5.1, the proposed NN-SCF scheme reduces the number of expensive Fourier transforms by $7\times$ compared to ICF and $3\times$ compared to classical SCF. The bulk of the remaining calculations are converted from complex arithmetic to highly parallelizable, low-cost real additions and multiplications.

5.3 System Implementation

To validate the theoretical advantages of the NN-SCF mapper, a simulation pipeline was developed using Python, NumPy, and PyTorch. The simulation environment parameters were configured to replicate the standard benchmarks used in the literature, as detailed in Table 5.2.

TABLE 5.2: Simulation and system parameters for the NN-SCF pipeline.

Parameter	**Value**
Subcarriers (N)	256
Oversampling Factor (J)	$4 \implies N_{\text{fft}} = 1024$
Cyclic Prefix (CP) Length	64
Clipping Ratio (CR)	6 dB
ICF Iterations (L) / SCF Target (k)	3
Modulation Schemes	QPSK, 16-QAM
Data Batch Size	$10{,}000$ symbols

5.3.1 Dataset Generation and Normalization

The dataset was generated by transmitting random bitstreams modulated under QPSK or 16-QAM schemes. Oversampling with a factor of $J = 4$ was enforced by padding the frequency-domain symbol vector with 768 guard subcarriers before taking the 1024-point IFFT. This is physically necessary because Nyquist-rate sampling ($J = 1$) underestimates the analog peaks and ignores peak regrowth.

The generated time-domain signal x_n served as the training input. The target (ground truth) was obtained by passing x_n through the classical SCF algorithm with $k = 3$ virtual iterations. Both the inputs and targets were split into real and imaginary components.

To accelerate backpropagation and match the bounded range of the triangular activation function, the real and imaginary components were normalized to the range $[-1, +1]$ using Min-Max scaling:

$$X_{\text{norm}} = 2 \cdot \frac{X_{\text{raw}} - X_{\text{min}}}{X_{\text{max}} - X_{\text{min}}} - 1$$

where X_{min} and X_{max} were computed across the training batch and saved as metadata to perform exact physical reconstruction during testing:

$$Y_{\text{pred}} = \left(\frac{Y_{\text{pred,norm}} + 1}{2} \right) (Y_{\text{max}} - Y_{\text{min}}) + Y_{\text{min}}$$

5.3.2 Neural Network Training Details

The custom PyTorch model implements the 1–2–1–1 topology described in Section 5.2.

While the original paper [10] recommends the Levenberg-Marquardt optimizer, standard deep learning libraries like PyTorch lack native, efficient support for second-order Marquardt optimization on large datasets. Consequently, we utilized the **Adam optimizer** with a learning rate of 0.001. The networks were trained independently to minimize the Mean Squared Error (MSE) loss:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N_{tr}} \sum_{i=1}^{N_{tr}} (y_i - \hat{y}_i)^2$$

Both networks were trained for 100 epochs using 70 training packages, with validation loss monitored on 10 validation packages at the end of each epoch to prevent overfitting.

5.4 Simulation Results and Analysis

After training, the saved PyTorch model weights were loaded to evaluate the model's inference performance on the remaining 20 testing packages.

5.4.1 Cubic Metric (CM) Performance

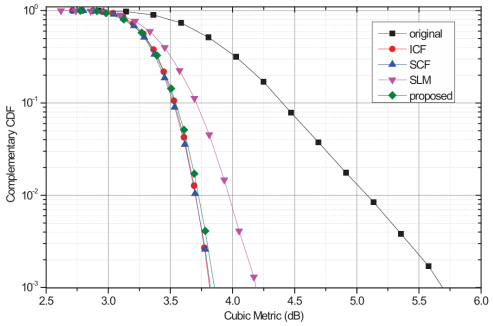
The Cubic Metric (CM) represents the power envelope fluctuations of the OFDM signal more accurately than PAPR when designing power amplifiers, as it accounts for the cubic non-linearities of the active components. The raw cubic metric (RCM) is defined as:

$$\text{RCM} = 20 \log_{10} \left(\text{rms} \left[\left(\frac{|x_n|}{\text{rms}(x_n)} \right)^3 \right] \right)$$

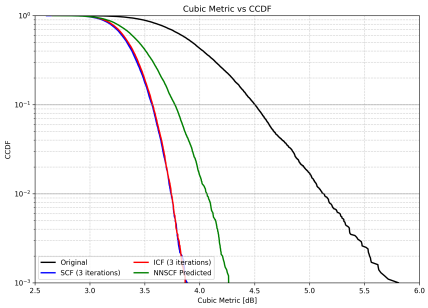
The standard CM in decibels is calculated relative to a reference signal:

$$\text{CM} = \frac{\text{RCM} - \text{RCM}_{\text{ref}}}{K}$$

where $\text{RCM}_{\text{ref}} = 1.52$ dB represents the RCM of a reference WCDMA signal, and $K = 1.56$ is an empirical scaling factor [10].



(A) Results from the original paper [10]



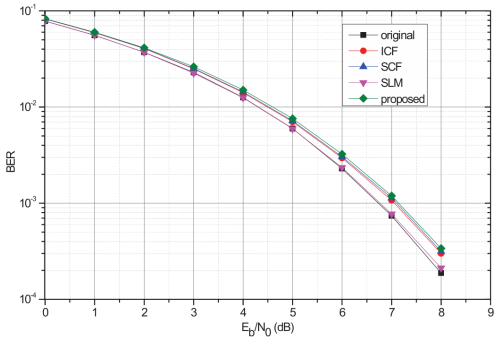
(B) Our simulated results

FIGURE 5.2: CCDF of the Cubic Metric (CM) for QPSK modulated OFDM signals ($N = 256$), comparing (a) original results [10] and (b) our PyTorch reproduction. The plot evaluates the capacity of the NN-SCF mapper to reproduce classical SCF noise compensation.

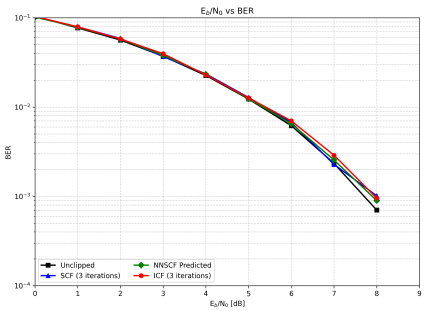
Figure 5.2 compares the Complementary Cumulative Distribution Function (CCDF) of the Cubic Metric for QPSK modulation. Our PyTorch implementation successfully reproduces the curves reported by the original authors. The NN-SCF curves lie within 0.1 dB of the classical SCF baseline at a CCDF of 10^{-3}, representing a significant 1.3 dB reduction in envelope fluctuation compared to original OFDM. This confirms that the lightweight neural network has accurately captured the complex, non-linear clipping noise cancellation.

5.4.2 Bit Error Rate (BER) Performance

Next, the Bit Error Rate (BER) performance was simulated over an Additive White Gaussian Noise (AWGN) channel to evaluate signal reconstruction accuracy.



(A) Results from the original paper [10]

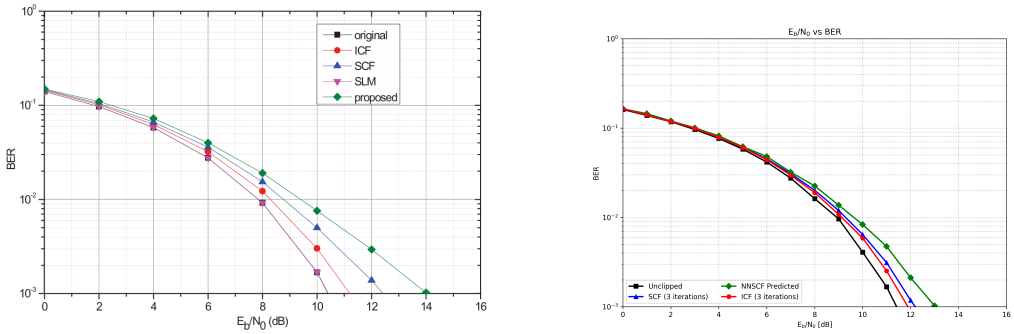


(B) Our simulated results

FIGURE 5.3: Bit Error Rate (BER) comparison over an AWGN channel for QPSK modulation ($N = 256$), comparing (a) original results [10] and (b) our simulated reproduction. The plot evaluates signal reconstruction quality under different peak-limiting frameworks.

As shown in Figure 5.3, the BER curves of all techniques for QPSK are virtually identical. The NN-SCF mapper successfully limits the peak power without introducing any additional in-band distortion, tracking the theoretical original OFDM BER down to 10^{-4} at $E_b/N_0 = 8$ dB.

Figure 5.4 compares the BER performance for the 16-QAM modulation scheme.



(A) Results from the original paper [10]

(B) Our simulated results

FIGURE 5.4: Bit Error Rate (BER) comparison over an AWGN channel for 16-QAM modulation ($N = 256$), comparing (a) original results [10] and (b) our simulated reproduction. The plot highlights the performance gap under higher-order amplitude modulation schemes.

Unlike QPSK, the 16-QAM results reveal a small performance gap. The NN-SCF curves exhibit an SNR penalty of approximately 1.5 dB at a target BER of 10^{-3} compared to classical SCF. This is because 16-QAM utilizes multiple amplitude levels. The increased constellation complexity makes it more difficult for a highly restricted 1–2–1–1 neural network to map the exact non-linear boundaries, causing minor frequency-domain information loss. However, the system retains a significant complexity advantage over the classical benchmarks.

5.5 Limitations

Despite its advantages, several limitations were identified during the implementation:

- **Lack of Spatial Correlation**: The MLP processes the signal sample-by-sample, meaning it cannot capture the temporal correlations introduced by the frequency-domain filter.
- **Configuration Overfitting**: The weights are trained for a specific clipping ratio, modulation scheme, and oversampling rate. Any change in the environment parameters requires retraining.

- **Optimization Constraint**: The PyTorch model had to be trained using the first-order Adam optimizer rather than the second-order Levenberg-Marquardt algorithm proposed in the literature, which restricted training convergence speed.
- **Omitted Parameters**: Several parameters were not given in the research paper, which restricted the ability to exactly replicate its results.
- **Model Capacity**: The highly compact 1–2–1–1 topology limits mapping accuracy for higher-order constellations like 64-QAM or 256-QAM.

5.6 Future Work

Future research can address these challenges through the following extensions:

- **Temporal Architectures**: Replacing the MLP with Convolutional Neural Networks (CNNs) or Recurrent Neural Networks (RNNs) to capture sample-to-sample memory and better emulate filter characteristics.
- **Generalized Training**: Training the model across a wide range of clipping ratios and modulation levels to create a single, robust physical-layer block.
- **Subcarrier Scaling**: Expanding the pipeline to evaluate performance and complexity gains on larger subcarrier systems ($N = 1024$ or 2048).
- **Parameter Optimization**: Having more liberty in choosing the parameters to make sure the mapper gets to the optimum results.

APPENDIX A

Glossary of Deep Learning Terms

This appendix provides concise definitions of deep learning concepts referenced throughout the document, intended as a quick reference for readers with a communications background.

Neuron

The basic computational unit of a neural network. It computes a weighted sum of its inputs, adds a bias, and applies an activation function: $y = f(\boldsymbol{w}^T \boldsymbol{x} + b)$.

Weight

A learnable parameter in a neural network that scales the connection between two neurons. The collection of all weights (and biases) constitutes the model's trainable parameters, denoted $\boldsymbol{\theta}$.

Bias

A learnable constant added to the weighted sum in a neuron before the activation function is applied: $z = \boldsymbol{W}\boldsymbol{x} + \boldsymbol{b}$. It allows the network to shift the activation function, giving it more flexibility.

Activation Function

A nonlinear function applied element-wise to the output of a neural network layer. Common choices include ReLU, sigmoid, tanh, softmax, and the linear (identity) function. Without nonlinear activations, a multi-layer network would collapse to a single linear transformation.

Fully Connected (FC) Layer

A layer where every neuron is connected to every neuron in the previous layer via a weight matrix $\boldsymbol{W}$. Also called a dense layer. The computational cost of an FC layer scales as $\mathcal{O}(n_{\text{in}} \times n_{\text{out}})$.

Forward Pass

The computation that propagates input data through the network, layer by layer, to produce an output. During inference (deployment), only the forward pass is needed—backpropagation is not required.

Inference

The process of using a trained model to make predictions on new, unseen data.

In the PAPR context, inference means performing one forward pass per OFDM symbol at real-time rates.

Loss Function

A scalar function that quantifies how far the network's output is from the desired output. Training minimises this function. Common choices include MSE for regression and cross-entropy for classification. In PAPR, custom losses combine PAPR and MSE terms.

Training

The iterative process of adjusting network parameters to minimise the loss function. Involves repeated forward passes, loss computation, backpropagation, and parameter updates over many epochs.

Gradient

The vector of partial derivatives of the loss function with respect to each trainable parameter. It indicates the direction and magnitude of the steepest increase in loss; the optimizer updates parameters in the opposite direction.

Backpropagation

The algorithm used to compute gradients of the loss function with respect to all network parameters by applying the chain rule of calculus, layer by layer, from the output back to the input. These gradients are then used by the optimizer to update the weights.

Optimizer

The algorithm that updates network parameters based on computed gradients. SGD (Stochastic Gradient Descent) is the simplest; Adam (Adaptive Moment Estimation) is the most commonly used in practice, combining momentum and per-parameter learning rate adaptation.

Learning Rate (η)

A hyperparameter that controls the step size of each parameter update. Too large a value causes training to diverge; too small a value makes convergence extremely slow.

Epoch

One complete pass through the entire training dataset. Training typically requires hundreds of epochs before the network converges to a good solution.

Batch

A subset of the training data processed together in one forward and backward pass. Using batches (rather than the entire dataset) enables efficient GPU utilisation and introduces beneficial stochastic noise into training.

Convergence

The point during training at which the loss function stabilises and further epochs yield negligible improvement. A model that fails to converge may be underfitting (too simple) or have a learning rate that is too high.

Hyperparameter

A configuration value set before training begins and not learned from data. Examples include learning rate, batch size, number of layers, number of epochs, and the loss balance factor α.

Overfitting

When a network learns to memorise the training data rather than generalise to unseen data. Symptoms include very low training loss but high validation/test loss. Mitigated by dropout, regularisation, and early stopping.

Regularisation

Any technique that prevents overfitting by constraining the model's complexity. Examples include dropout, weight decay (L_2 regularisation), and batch normalization.

Dropout

A regularisation technique that randomly sets a fraction of neuron outputs to zero during each training step. This prevents neurons from co-adapting and encourages the network to learn redundant representations, reducing overfitting.

Batch Normalization (BatchNorm)

A technique that normalises the inputs to each layer by subtracting the batch mean and dividing by the batch standard deviation. This stabilises training, allows higher learning rates, and acts as a mild regulariser.

Vanishing Gradients

A problem where gradients become extremely small as they propagate through many layers during backpropagation, causing early layers to learn very slowly or not at all. ReLU activations and residual connections help mitigate this.

Autoencoder

A neural network architecture consisting of an encoder that maps input data to a lower-dimensional (or transformed) representation, and a decoder that reconstructs the original data. In the PAPR context, the encoder produces a low-PAPR signal and the decoder recovers the original data symbols.

References

[1] B. S. d. C. da Silva, V. D. Souto, R. D. Souza, and L. L. Mendes, "A survey of papr techniques based on machine learning," *Sensors*, vol. 24, no. 6, p. 1918, 2024.

[2] M. Kim, W. Lee, and D.-H. Cho, "A novel papr reduction scheme for ofdm system based on deep learning," *IEEE Communications Letters*, vol. 22, no. 3, pp. 510–513, 2017.

[3] Z. Liu et al., "Low-complexity papr reduction method for ofdm systems based on real-valued neural networks," *IEEE Wireless Communications Letters*, 2020. DOI: 10.1109/LWC.2020.3005656

[4] J. Cioffi, "Par reduction in multicarrier transmission systems," 1998.

[5] Q. Si, B. Wang, and M. Jin, "A novel tone reservation scheme based on deep learning for papr reduction in ofdm systems," *IEEE Communications Letters*, vol. 24, no. 6, pp. 1271–1274, 2020.

[6] N. Shlezinger, J. Whang, Y. C. Eldar, and A. G. Dimakis, "Model-based deep learning," *Proceedings of the IEEE*, 2023.

[7] L. Li, C. Tellambura, and X. Tang, "Improved tone reservation method based on deep learning for papr reduction in ofdm system," in *2019 11th International Conference on Wireless Communications and Signal Processing (WCSP)*, 2019, pp. 1–6. DOI: 10.1109/WCSP.2019.8928103

[8] X. Wang, N. Jin, and J. Wei, "A model-driven deep learning algorithm for papr reduction in ofdm system," *IEEE Communications Letters*, 2021.

[9] L. Wang and C. Tellambura, "A simplified clipping and filtering technique for par reduction in ofdm systems," *IEEE signal processing letters*, vol. 12, no. 6, pp. 453–456, 2005.

[10] I. Sohn and S. C. Kim, "Neural network based simplified clipping and filtering technique for papr reduction of ofdm signals," *IEEE Communications Letters*, vol. 19, no. 8, pp. 1438–1441, 2015.